# Spatio-temporal Land Cover Classification with UK Sentinel-2 Time Series

## 1 Introduction

Benchmark datasets play a pivotal role in advancing machine learning for Earth Observation (EO), enabling systematic development and evaluation of algorithms across diverse applications such as land cover, agricultural [1, 15] classification and cloud removal [3, 4]. Despite their importance, existing land cover benchmarks vary considerably in their labelling strategies. For example, one [2] generates labels from consensus scores across human annotators, whereas another [13] performs global semantic segmentation using broad thematic classes *(e.g. road, water, tree)*, favouring generalisability over thematic precision.

The Sen4Map [11] dataset addresses a notable gap in this landscape by linking high-resolution Sentinel-2 time series with in-situ observations from the LUCAS (Land Use and Cover Area frame Survey) campaign. Unlike labels derived exclusively from remote sensing, LUCAS provides harmonised, field-based ground truth collected across the EU. This integration of field-collected data with dense temporal imagery supports robust supervised classification while preserving alignment to downstream policy frameworks.

### 1.1 Study Objectives and Experimental Setup

This study assesses the generalisation capacity of land cover models when training is **limited to the UK subset of the Sen4Map data**. Compared to the original EU-wide benchmark [11], we examine how reduced geographic diversity affects performance and whether additional spatial features and covariates can mitigate this drop.

To evaluate this, we revisit and reformulate the benchmark along three analytical dimensions:

i. **Spatial contextualisation:** We introduce two scenarios of spatial feature sets to capture spatially aware information:
   - k-nearest neighbours
   - fixed radius median aggregation

ii. **Model diversification:** In addition to Random Forests (RF), we implement Support Vector Machines (SVM) and XGBoost (XGB) — two widely adopted algorithms in remote sensing due to their ability to capture non-linear feature interactions and maintain strong performance in high-dimensional settings [8, 14].

iii. **Baseline comparison:** We compare UK-only models to the original EU+UK Random Forest baseline, quantifying performance loss from geographic constraint and testing whether spatial features can compensate.

### 1.2 Data Description

*1.2.1 Sentinel-2 Imagery.* 64×64 pixels images are extracted from Level-1C Sentinel-2A and 2B tiles, aligned with LUCAS survey points. Each patch forms a multi-temporal sequence in 2018 *(aligned with the last available LUCAS Land Cover data for the UK) – see Fig.1*.

Among Sentinel-2's image bands, the 60m atmospheric bands *(e.g., B01-Aerosols, B09-Water Vapour)* are excluded in this study, as they are primarily intended for atmospheric correction. Only the 10m and 20m bands are retained, given their relevance for land cover mapping.

The final set of input bands include B3, B4, B5, B6, B77, B8, B8A, B11, B12 along with the Scene Classification Layer (SCL). The SCL mask is used to exclude pixels affected by clouds, cirrus and snow (`'SCL'<9`) [16].

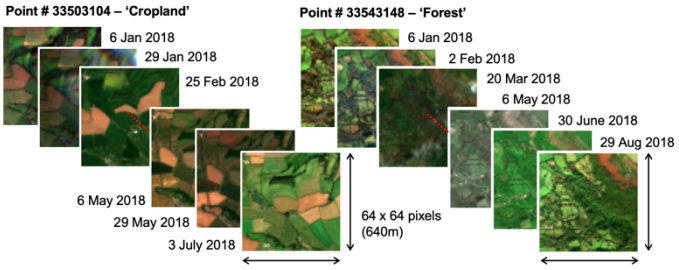


***Fig. 1**: Snapshot examples of 2 Sentinel-2 time series from the Sen4Map benchmark dataset. Each series is associated with a LUCAS point denoting 'Cropland' and another 'Forest' respectively.*

*1.2.1 LUCAS Land Cover.* The LUCAS dataset is a harmonised, EU-wide field survey coordinated triennially by Eurostat [17], which records land cover and other environmental attributes at georeferenced sites. Each observation metadata includes GPS coordinates – the theoretical centre point used to extract the corresponding Sentinel-2 image patch.

This study utilised only the primary land cover classification `'lc1'` as ground truth. Although a small subset of samples (n=70) includes a secondary land cover label `'lc2'` with estimated percentage coverage, only eight cases report `lc2` as the dominant class. These cases all correspond to points on sparser agricultural sites (e.g., apple orchards over grasslands) and are thus retained as originally surveyed without relabelling.

To improve model interpretability and mitigate class imbalance, labels are reclassified into nine high-level land cover categories, based on the UN Land Cover Classification System (LCCS) [6] – *for detailed mapping see Appendix*. This harmonisation enhances compatibility with prior studies and policy-relevant land monitoring frameworks.

## 2 Exploratory Spatio-temporal Data Analytics

### 2.1 Density Estimation

*2.1.1 Temporal Density.* While Sentinel-2 nominally offers a five-day revisit cycle, the effective temporal resolution is substantially reduced by cloud contamination, snow cover and high cirrus reflectance. To improve temporal consistency and minimise noise, we filtered for Sentinel-2 tiles with reported cloud cover below 40%, resulting in uneven time series lengths across LUCAS points – *see Fig 2*.

To address this variability, we applied a monthly aggregation strategy similar to [12], computing the median reflectance for each spectral band across all valid observations within each month, zeroing all values of the month if no cloud-free images are available for that month. This produces a 12-step time series per point and reduces the influence of outliers while preserving seasonal dynamics.

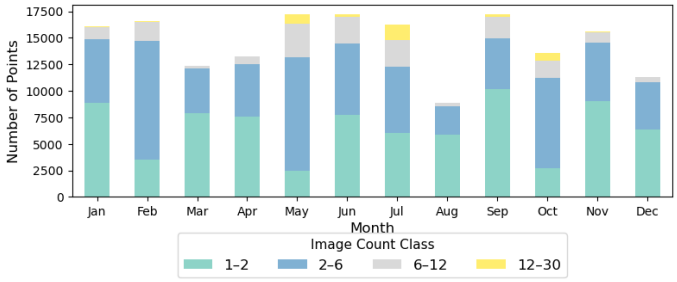


***Fig 2**. Histogram of valid monthly observations per LUCAS point. The groupings were determined through Jenks Natural Breaks.*

*2.1.2 Spatial Density.* The spatial distribution of samples exhibits significant class imbalance – *Fig 3*, posing challenges for model training and evaluation. Grassland dominates with approximately 48% of observations, while cropland comprises just 17%. The remaining classes are even more sparsely represented. Although this reflects the actual landscape composition of the UK, it limits classifier performance – particularly for minority classes. Given the scope of this exercise, we retain the natural class distribution to preserve alignment with real-world frequences, while acknowledging its impact on the accuracy of underrepresented classes in our interpretation of results.

Data leakage due to overlapping satellite imagery is a well-documented challenge in remote sensing machine learning, often leading to inflated performance estimates [7, 10]. To mitigate this, we computed pairwise distances between all LUCAS points to verify that image patches were spatially independent. Each patch spans 640m (64 x 10m pixels), so nearby samples could potentially overlap and share redundant information.

To ensure sufficient spatial separation, we verified that all point pairs exceeded twice the patch size (1280m) – *see Fig.4*. This confirms that all patches are spatially distinct, reducing the risk of data leakage and enabling a more robust assessment of model generalisation.

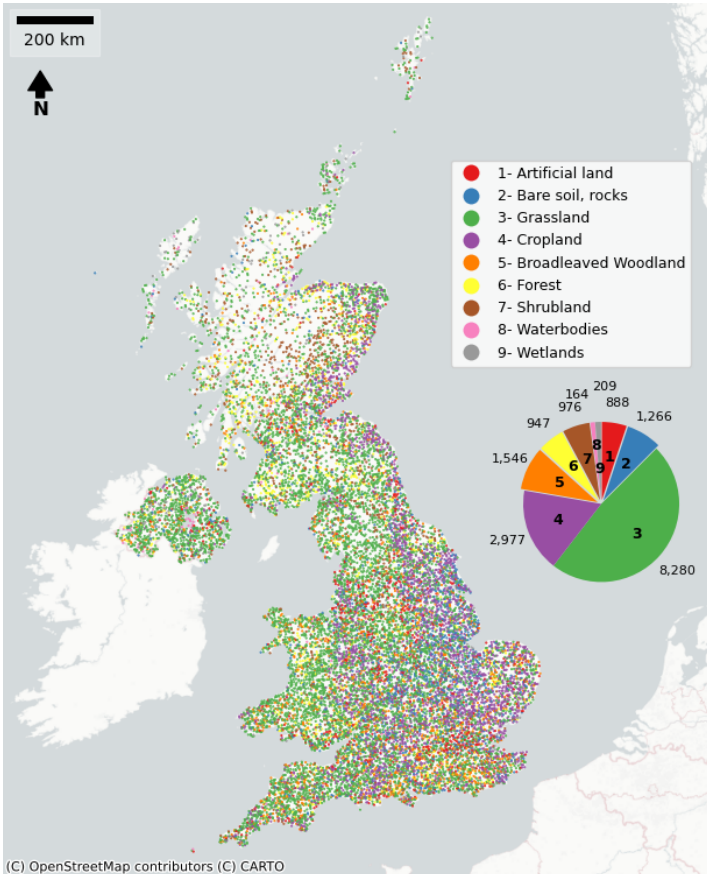


***Fig 3**: LUCAS point distribution across the UK, with class frequency pie chart in inset. There is observable spatial autocorrelation in land cover – for e.g., cropland (purple, label 4) clusters more heavily in eastern and southeastern England.*

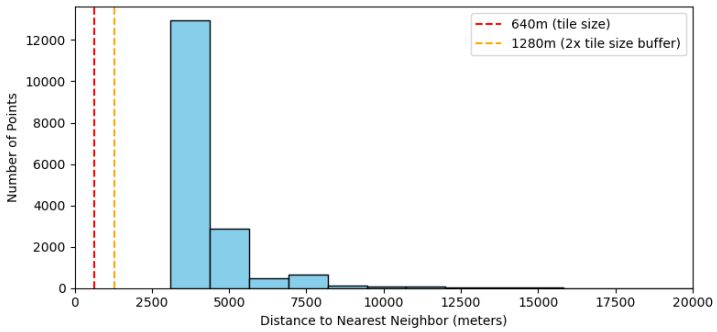


***Fig 4**. Nearest neighbour distance distribution for all UK points. The minimum distance between any two samples measures 3104.67m.*

### 2.2 Autocorrelation Analysis

To determine an appropriate fixed radius for aggregating neighbouring reflectance values, we conducted spatial autocorrelation analysis on the `lc1` label. Specifically, we computed Moran's I and Geary's C statistics across distance bands of 4 to 10km, capturing the degree of spatial clustering in categorical land cover patterns, detailed in *Fig 5*.

The results show consistent and statistically significant positive spatial autocorrelation across all distance tested (p-value<0.01 for both cases) – *see Fig.5.* The slight rise in Geary's C beyond 5km suggests a weakening of spatial similarity at larger distances – we thus selected a 5km radius for

spatial aggregation. This choice strikes a balance between capturing local homogeneity and meaningful clustering, while ensuring sufficient neighbouring points for stable aggregation.

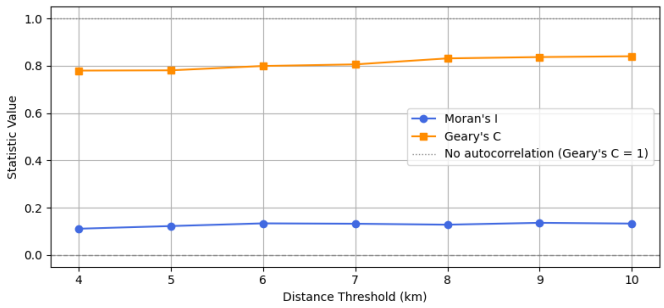


*Fig 5. Moran's I and Geary's C across 4-10km distance bands, where p-value<0.005 in both cases.*

## 3 Methodology

This study evaluates the Sen4Map land cover classification benchmark by restricting training to the **United Kingdom subset**, in contrast to the original EU-wide model [11]. Our aim is to evaluate whether incorporating spatial context and metric-aware hyperparameter tuning can offset the performance loss that typically results from training on a geographically narrower and class-imbalanced dataset.

### 3.1 Model Selection

Each model contributes distinct strengths to this comparative framework:

- **Random Forest (RF)** is an ensemble of decision trees, robust to noise and overfitting, and widely used in EO due to its strong performance with limited tuning [5]. It handles non-linearity well and is relatively interpretable.
- **Support Vector Machine (SVM)** excels in high-dimensional spaces and is particularly effective when data is not linearly separable. Its reliance on kernel functions allows it to capture complex class boundaries, though it can be sensitive to class imbalance and requires careful feature scaling.
- **XGBoost** is a gradient-boosted tree algorithm that incrementally correct residual errors. It offers strong regularisation capabilities, handles sparse data efficiently and consistently outperforms other models in structured classification tasks.

These models are evaluated using identical feature sets and experimental protocols.

### 3.2 Data and Feature Engineering

*3.2.1 Baseline Representation.* The baseline feature set comprises a 12-step temporal feature profile for each LUCAS point, where each step represents the monthly median reflectance per Sentinel-2 band. Cloud-contaminated pixels were masked using the Scene Classification Layer (SCL) to avoid noise from clouds, shadow and snow. Months without valid observations were zero-filled, following Sentinel-2 processing protocol [12, 16]

The target variable is the primary land cover class `lc1`, reclassified into nine high-level categories based on the UN LCCS [6]. This reduces label granularity and mitigate extreme class imbalance by grouping semantically similar subclasses.

Standard classifiers do not explicitly account for spatial dependence, aside from implicit correlations between neighbouring pixels. However, as shown in our spatial autocorrelation analysis (Section 2.2), land cover labels in our dataset exhibit statistically significant clustering over space.

*3.2.2 Spatial Feature Augmentation.* To capture spatial context beyond individual point observations, we evaluate two strategies for generating additional spatial covariates:

- **k-Nearest Neighbours (kNN)**: For each LUCAS point, we appended the reflectance values of its 10 nearest neighbours, identified using haversine distances. This spatial feature augmentation approach, adapted from [9], expands the feature vector elevenfold relative to the original input.
- **Fixed-Radius Median Aggregation (5km):** For each point, we computed the monthly median reflectance across all neighbouring points within a 5km radius *(as determined in Section 2.2).* This method aligns with temporal harmonisation practices in Sentinel-2 workflows [12] and aims to capture local spatial structure while smoothing fine-grained variability.

*3.2.3 Data Partitioning and Class Balance.* The dataset was partitioned into training (70%), validation (15%) and test (15%) sets. To preserve the natural class distribution and spatial heterogeneity of the landscape, no resampling or rebalancing techniques were applied. This partitioning strategy was applied consistently across all three classifiers (RF, SVM and XGB) to ensure comparability of performance.

### 3.3 Evaluation Metrics

Given the extreme class imbalance (e.g., grassland ~48%, minority classes <5%), we selected:

- **Macro F1** as the primary metric for model selection and optimisation. By equally weighting each class's F1 score, it penalises imbalanced performance and support a fairer assessment of model performance in imbalanced settings.
- **Macro recall** and **weighted F1** as secondary metrics, offering complementary views on class sensitivity and overall correctness.

This approach departs from the original Sen4Map benchmark, where overall accuracy was sufficient due to more balanced EU & UK-wide coverage. In

this study's case, accuracy would be misleading – masking poor performance on rarer class types.

### 3.4 Hyperparameter Optimisation

To tune model parameters, we employed a two-stage strategy:

1. GridSearchCV for structured baseline tuning
2. Optuna (Bayesian optimisation) for adaptive, metric-driven search

Grid search is exhaustive and interpretable, but inefficient in high-dimensional spaces, and biased when using accuracy as an objective. In contrast, Optuna models the objective using Tree-structured Parzen Estimators (TPE), enabling efficient exploration of promising parameter regions. Critically, Optuna supports direct optimisation for macro F1, aligning tuning to the evaluation goal. The number of trials was fixed at 50 for each model.

#### *3.4.1 Model-Specific Configurations.*

***RF**:* Grid search provided a reasonable baseline for tuning `n_estimators`, `max_depth`, `min_samples_split`, and `min_samples_leaf`. However, RF exhibited a strong tendency to overfit majority classes. Optuna extended the search across continuous ranges and explicitly optimised for macro F1, improving minority class sensitivity.

***SVM*** required careful tuning due to its sensitivity to feature scaling and non-linear parameter interactions. Grid search explored `C`, `gamma`, kernel type and class weights, but struggled with the expanded feature space. Optuna's additional log-uniform sampling of `C` and `gamma` was better-suited to SVM's exponential parameter landscape. Input features were standardised using StandardScaler.

***XGB*** was the most sensitive to hyperparameter configuration and benefited significantly from Bayesian optimisation tuning. Grid search over standard parameters (`n_estimators`, `max_depth`, `learning_rate`, `colsample_bytree` and `subsample`) delivered marginal gains, likely due to coarse search granularity. Optuna extended the search to include regularisation (`gamma`, `reg_lambda`) and `min_child_weight`, offering finer control over feature sampling.

## 4 Results

### 4.1 Impact of Spatially Contextualised Features

To evaluate the impact of spatial context, we tested three input configurations for RF, SVM and XGB: (i) point-based baseline, (ii) 5km median aggregation; and (iii) 10-nearest neighbours augmentation.

Across all models, the 5km median aggregation of neighbours improved both macro F1 and overall accuracy compared to the baseline, confirming the value of spatial context in a geographically constrained setting – *see table A*. However, the 10-nn approach did not outperform the baseline, particularly for RF and SVM, where performance declined considerably – suggesting that high-dimensional neighbour concatenation may introduce noise or redundancy.

***Table A.*** *Impact of Spatial Contextualisation on Model Performance*

| Model | | Baseline | Median of Neighbours | 10-Nearest Neighbours |
|---|---|---|---|---|
| **RF** | *Accuracy* | 0.705 | 0.706 | 0.677 |
| | *Macro F1* | 0.499 | 0.493 | 0.445 |
| **SVM** | *Accuracy* | 0.710 | 0.713 | 0.651 |
| | *Macro F1* | 0.471 | 0.486 | 0.380 |
| **XGBoost** | *Accuracy* | 0.714 | 0.720 | 0.704 |
| | *Macro F1* | 0.516 | 0.549 | 0.490 |

### 4.2 Per-Class Model Performance (5km Median)

*Table B* summarises per-class metrics. Grassland and Cropland, the most prevalent classes, are consistently well-classified across all models, reflecting clear phenological signals and high representation in the training data.

The most pronounced differences emerge in minority and mid-frequency classes such as Bare soil, Shrubland, and Wetlands. Here, XGBoost demonstrates clear advantages in recall, outperforming RF and SVM. Improvements in these classes are marginal in RF and largely absent in SVM—highlighting the benefit of boosting methods coupled with spatial features and macro F1 tuning in imbalanced EO classification settings.

The confusion matrices *(Fig. 6)* reinforce these patterns. All models agree closely on dominant classes but diverge on rarer types. Notably:

- Bare soil and Shrubland are often misclassified as Grassland, especially by RF and SVM – likely due to spectral similarity and training imbalance. *4.2.1* examines these labels more closely.
- Broadleaved and Forests are frequently confused, suggesting that monthly composites may not fully capture key phenological differences like leaf emergence or change.
- Wetlands are rarely predicted correctly across all models – indicating that spatial context alone is insufficient when class support is extremely low.

#### *4.2.1 Error Analysis by Class (Labels 2 and 7).*

Spatial prediction maps *(Fig 7 and 8)* highlight mis-classification patterns for the two lowest-performing classes: Label 2 (Bare soil) & Label 7 (Shrubland).

For Bare soil, all models struggle, but XGB correctly predicts a higher number of spatially dispersed sites (44 vs 18 in RF, 27 in SVM), suggesting stronger generalisation beyond dense clusters. This likely reflects XGB's iterative error-driven learning, which prioritises misclassified or outlier classes during training.

***Table B.*** *Per-Class Precision, Recall and F1-score for RF, SVM and XGB Models Using 5km Median Aggregation*

| Classes | | Random Forest | | | Support Vector Machine | | | XGBoost | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | *n* | *Precision* | *Recall* | *F1-score* | *Precision* | *Recall* | *F1-score* | *Precision* | *Recall* | *F1-score* |
| 1 Artificial land | 135 | 0.559 | 0.563 | 0.561 | 0.579 | 0.681 | 0.626 | 0.604 | 0.644 | 0.624 |
| 2 Bare soil | 193 | 0.562 | 0.093 | 0.160 | 0.435 | 0.140 | 0.212 | 0.400 | 0.228 | 0.290 |
| 3 Grassland | 1247 | 0.714 | 0.925 | 0.806 | 0.741 | 0.899 | 0.813 | 0.763 | 0.896 | 0.824 |
| 4 Cropland | 462 | 0.732 | 0.764 | 0.748 | 0.750 | 0.818 | 0.783 | 0.765 | 0.766 | 0.765 |
| 5 Broadleaves | 234 | 0.665 | 0.500 | 0.571 | 0.634 | 0.547 | 0.587 | 0.610 | 0.547 | 0.577 |
| 6 Forest | 135 | 0.699 | 0.533 | 0.605 | 0.705 | 0.548 | 0.617 | 0.718 | 0.548 | 0.622 |
| 7 Shrubland | 131 | 0.800 | 0.183 | 0.298 | 0.552 | 0.122 | 0.200 | 0.611 | 0.336 | 0.433 |
| 8 Waterbodies | 22 | 0.923 | 0.545 | 0.686 | 0.667 | 0.455 | 0.541 | 0.833 | 0.682 | 0.750 |
| 9 Wetlands | 29 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.200 | 0.034 | 0.059 |
| Weighted Ave. | 2588 | 0.691 | 0.706 | 0.667 | 0.671 | 0.713 | 0.680 | 0.699 | 0.720 | 0.702 |
| **Macro F1** | | | | **0.493** | | | **0.486** | | | **0.549** |
| **Accuracy** | | | | **0.706** | | | **0.713** | | | **0.720** |

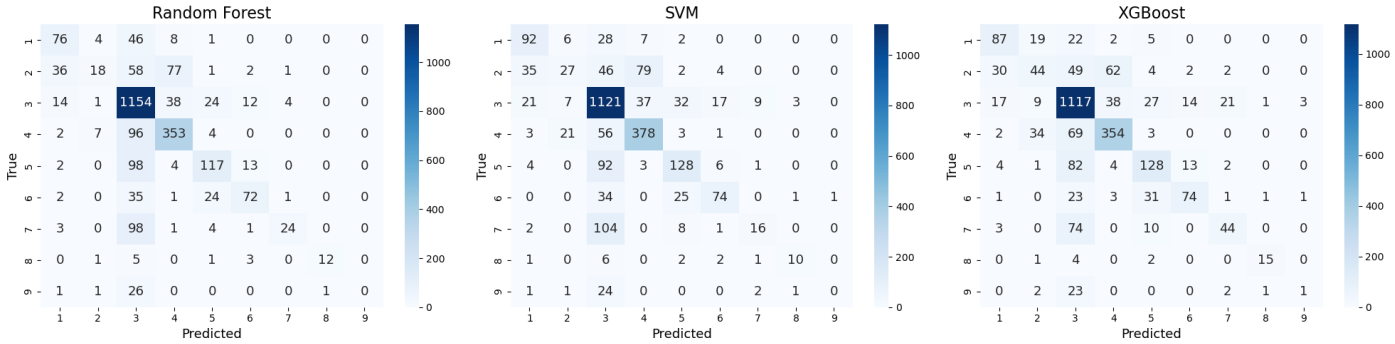


***Fig 6****. Confusion Matrices for Optuna-Tuned Models Using 5km Median Aggregation*

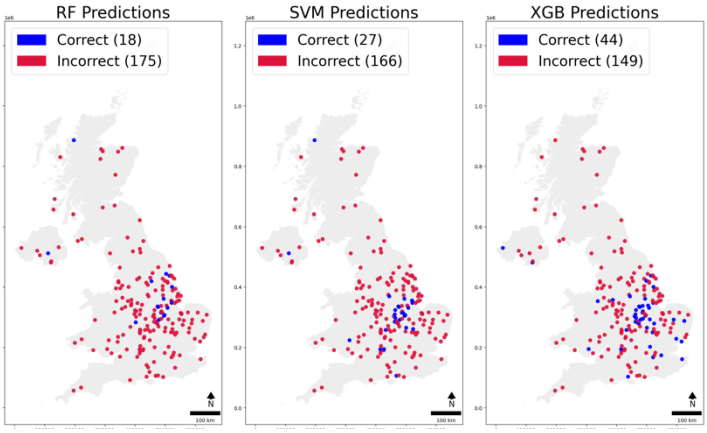


***Fig 7.*** *Predictions for Label 2 - Bare Soil/Rocks*

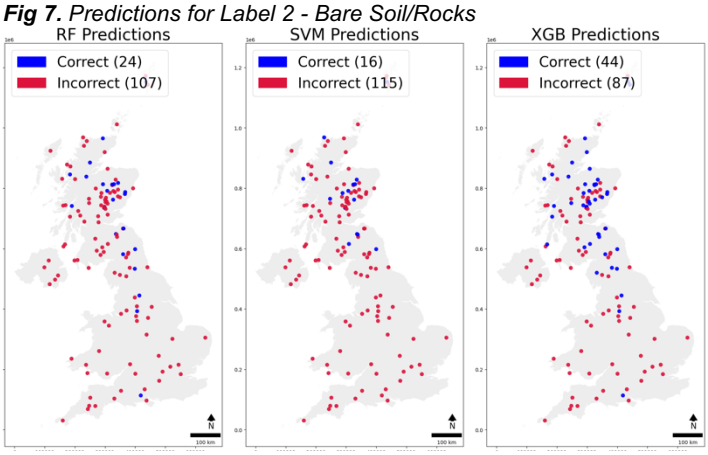


***Fig 8.*** *Predictions for Label 7 - Shrubland*

For Shrubland, SVM correctly identifies only 16 points, primarily within dense clusters, indicating poor generalisation beyond localised patterns. In contrast, XGB correctly classifies 44 points – capturing both clustered and dispersed instances more effectively – highlighting its superior ability to learn from sparse and spatially dispersed training signals.

### 4.3 Comparison to Full Sen4Map EU+UK Baseline

The original Sen4Map benchmark, trained on the full EU+UK dataset, achieved an overall accuracy of 0.66 and a weighted F1 of 0.67 using a Random Forest classifier - reflecting performance under broad geographic coverage and class diversity.

By contrast, our UK-only Random Forest model, augmented with 5km spatial aggregation, achieves comparable accuracy (0.706) and weighted F1 (0.667) – despite being trained on just 5.1% of the original sample size. This suggests that incorporating spatial context can substantially offset the reduced geographic coverage in data.

Our best performing model, XGBoost with 5km aggregation, further improves on both metrics, reaching 0.720 accuracy and delivering consistently higher F1 scores on minority and mid-frequency classes such as Shrubland, Artificial Land and Forest.

These results demonstrate that spatial contextualisation enables regional models to match – and in some cases exceed – the performance of continent-scale baselines, particularly when combined with more expressive algorithms and metric-aware optimisation.

## 5 Discussion

## 5.1 Model Comparison and Interpretability

Despite architectural differences, Random Forest, SVM and XGBoost achieved comparable overall metrics, particularly when enhanced with spatial aggregation. This convergence suggest that all three models were able to reach a stable generalisation optimum under the given feature representation.

However, XGB consistently outperformed both RF and SVM on macro-averaged metrics, particularly for underrepresented classes. While overall accuracy remained similar (~0.71-0.72), XGB achieved the highest macro F1 (0.549) – indicating better balance across class predictions.

This reflects XGB's strength in modelling complex non-linear relationships and handling class imbalance through boosting and internal regularisation mechanisms. Its compatibility with fine-grained hyperparameter tuning, particularly when paired with Bayesian optimisation, enables it to more effectively navigate trade-offs between bias and variance. This advantage is evident in Fig 7 and 8, where XGB successfully captures more spatially dispersed minority class instances.

Random Forest, while less performant, remains valuable for its interpretability. We assessed feature importance using permutation importance, which better handles feature correlation than the default Gini-based approach – a critical consideration given the higher inter-band correlation in Sentinel-2 data.

Permutation analysis revealed that the most informative features were from spring and early summer (April-July), especially the red-edge (B5-B8A) and shortwave infrared bands (B11-B12). These bands are sensitive to vegetation structure, chlorophyll content, and soil moisture – key indicators for distinguishing land cover types. Feature importance was clearly seasonally driven, aligning with peak vegetation activity and land management practices such as sowing and grazing.

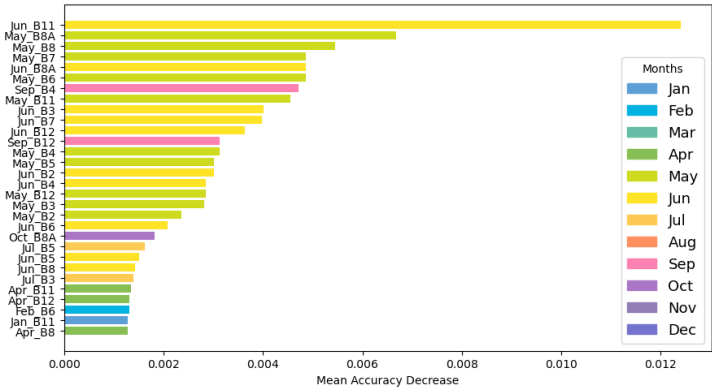


***Fig 9.*** *Top 30 Feature Permutation Importance*

SVM underperformed, particularly on minority classes. Its rigid decision boundaries, sensitivity to class imbalance, and lack of inherent probabilistic outputs likely contributed to weaker generalisation. While efficient in simpler or linearly separable tasks, SVM showed limited flexibility under spectral overlap and sparse class representation.

## 5.2 Spatial Generalisation Across Study Area

Model performance varied spatially, reflecting differences in landscape heterogeneity, class separability and label distribution. Classes with broad coverage and clear phenological signals - such as Grassland and Cropland – were consistently well classified. In contrast, rare or fragmented classes such as Wetlands, Shrubland or Bare soil showed high misclassification rates, often confused with dominant vegetated types

Spatial aggregation improved classification, especially for mid-frequency classes near land cover transitional zones, where local context supports more accurate classification. However, gains were limited for sparsely represented or spatially incoherent classes. For instance, wetlands remained poorly predicted despite contextual features, highlighting the importance of minimum sample thresholds for reliable generalisation.

Overall, these results reinforce a key insight: while spatial features help mitigate the effects of geographic constraint, generalisation remains constrained by class imbalance, spectral signal ambiguity and uneven spatial sampling density.

## 5.3 Limitations and Future Work

Several methodological limitations should be acknowledged. First, the fixed 5km aggregation radius does not account for landscape barriers, discontinuities or variable sampling density. Though effective as a first-order approximation, more nuanced spatial modelling – such as graph-based representations or geostatistical models – could better capture landscape structure.

Secondly, the use of 64x64 pixel patches may underrepresent broader land patterns. Larger tile aggregation. The original Sen4Map benchmark noted improved performance using 3×3 tile (192×192 pixels), which captures coarser-scale land patterns. This approach, however, requires significant preprocessing and is beyond the computational scope of this report. A comparative analysis between this and the adjacency-based method would be a valuable extension.

Finally, model performance is ultimately bounded by label quality. The LUCAS dataset, while highly standardised, includes a small number of ambiguous cases (e.g., mixed orchard/grassland sites), which were retained without reclassification. Inclusion of secondary land cover information (LC2) or uncertainty-aware labels could improve model calibration in future studies.

## References


[1] Steve Ahlswede, Christian Schulz, Christiano Gava, Patrick Helber, Benjamin Bischke, Michael Förster, Florencia Arias, Jörn Hees, Begüm Demir, and Birgit Kleinschmit. 2023. *TreeSatAI Benchmark Archive*: a multi-sensor, multi-label dataset for tree species classification in remote sensing. *Earth Syst. Sci. Data* 15, 2 (February 2023), 681–695. https://doi.org/10.5194/essd-15-681-2023

[2] Hamed Alemohammad and Kevin Booth. 2020. LandCoverNet: A global benchmark land cover classification training dataset. https://doi.org/10.48550/arXiv.2012.03111

[3] Cesar Aybar, Luis Ysuhuaylas, Jhomira Loja, Karen Gonzales, Fernando Herrera, Lesly Bautista, Roy Yali, Angie Flores, Lissette Diaz, Nicole Cuenca, Wendy Espinoza, Fernando Prudencio, Valeria Llactayo, David Montero, Martin Sudmanns, Dirk Tiede, Gonzalo Mateo-García, and Luis Gómez-Chova. 2022. CloudSEN12, a global dataset for semantic understanding of cloud and cloud shadow in Sentinel-2. *Sci. Data* 9, 1 (December 2022), 782. https://doi.org/10.1038/s41597-022-01878-2

[4] Patrick Ebel, Yajin Xu, Michael Schmitt, and Xiao Xiang Zhu. 2022. SEN12MS-CR-TS: A Remote-Sensing Data Set for Multimodal Multitemporal Cloud Removal. *IEEE Trans. Geosci. Remote Sens.* 60, (2022), 1–14. https://doi.org/10.1109/TGRS.2022.3146246

[5] Pall Oskar Gislason, Jon Atli Benediktsson, and Johannes R. Sveinsson. 2006. Random Forests for land cover classification. *Pattern Recognit. Lett.* 27, 4 (March 2006), 294–300. https://doi.org/10.1016/j.patrec.2005.08.011

[6] Martin Herold, R Hubald, and Antonio Di Gregorio. 2009. Translating and evaluating land cover legends using the uppercaseUN Land Cover Classification System (uppercaseLCCS). *GOFC-GOLD Rep. No 43* 43, (January 2009).

[7] Jakub Nalepa, Michal Myller, and Michal Kawulok. 2019. Validating Hyperspectral Image Segmentation. *IEEE Geosci. Remote Sens. Lett.* 16, 8 (August 2019), 1264–1268. https://doi.org/10.1109/LGRS.2019.2895697

[8] Jeongmook Park, Yongkyu Lee, and Jungsoo Lee. 2021. Assessment of Machine Learning Algorithms for Land Cover Classification Using Remotely Sensed Data. *Sens. Mater.* 33, 11 (November 2021), 3885. https://doi.org/10.18494/SAM.2021.3612

[9] Aleksandar Sekulić, Milan Kilibarda, Gerard B. M. Heuvelink, Mladen Nikolić, and Branislav Bajat. 2020. Random Forest Spatial Interpolation. *Remote Sens.* 12, 10 (January 2020), 1687. https://doi.org/10.3390/rs12101687

[10] Harald Semmelrock, Simone Kopeinik, Dieter Theiler, Tony Ross-Hellauer, and Dominik Kowald. 2023. Reproducibility in Machine Learning-Driven Research. https://doi.org/10.48550/arXiv.2307.10320

[11] Surbhi Sharma, Rocco Sedona, Morris Riedel, Gabriele Cavallaro, and Claudia Paris. 2024. Sen4Map: Advancing Mapping With Sentinel-2 by Providing Detailed Semantic Descriptions and Customizable Land-Use and Land-Cover Data. *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.* 17, (2024), 13893–13907. https://doi.org/10.1109/JSTARS.2024.3435081

[12] Giulio Weikmann, Claudia Paris, and Lorenzo Bruzzone. 2021. TimeSen2Crop: A Million Labeled Samples Dataset of Sentinel 2 Image Time Series for Crop-Type Classification. *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.* 14, (2021), 4699–4708. https://doi.org/10.1109/JSTARS.2021.3073965

[13] Junshi Xia, Naoto Yokoya, Bruno Adriano, and Clifford Broni-Bediako. 2022. OpenEarthMap: A Benchmark Dataset for Global High-Resolution Land Cover Mapping. https://doi.org/10.48550/arXiv.2210.10732

[14] Saiping Xu, Qianjun Zhao, Kai Yin, Feifei Zhang, Dingbang Liu, and Guang Yang. 2019. Combining random forest and support vector machines for object-based rural-land-cover classification using high spatial resolution imagery. *J. Appl. Remote Sens.* 13, 1 (February 2019), 014521. https://doi.org/10.1117/1.JRS.13.014521

[15] Sen4AgriNet: A Harmonized Multi-Country, Multi-Temporal Benchmark Dataset for Agricultural Earth Observation Machine Learning Applications | IEEE Conference Publication | IEEE Xplore. Retrieved March 31, 2025 from https://ieeexplore.ieee.org/document/9553603

[16] Sentinel-2. Retrieved March 31, 2025 from https://sentiwiki.copernicus.eu/web/sentinel-2

[17] LUCAS - Land use and land cover survey. Retrieved April 2, 2025 from https://ec.europa.eu/eurostat/statistics-explained/index.php?title=LUCAS_-_Land_use_and_land_cover_survey

## Appendix

*Table C: LUCAS Code mapping to LCCS Label*

| LCCS Label | LUCAS Code | LUCAS Description |
|---|---|---|
| 1 (artificial land) | A00 | Artificial land |
| | A10 | Roofed built-Up areas |
| | A11 | Buildings with 1 to 3 floors |
| | A12 | Buildings with more than 3 floors |
| | A13 | Greenhouses |
| | A20 | Artificial non-built-up area |
| | A21 | Non-built-up area features |
| | A30 | Other artificial areas |
| 2 (bare soil, rocks) | A22 | Non-built-up linear features |
| | F00 | Bare land and lichens/moss |
| | F10 | Rocks and stones |
| | F20 | Sand |
| | F40 | Other bare soil |
| 3 (grassland) | B50 | Fodder crops |
| | B51 | Clovers |
| | B52 | Lucerne |
| | B53 | Other leguminous and mixtures for fodder |
| | B54 | Mixed cereals for fodder |
| | B55 | Temporary grasslands |
| | E00 | Grassland |
| | E10 | Grassland with sparse tree/shrub cover |
| | E20 | Grassland without tree/shrub cover |
| | E30 | Spontaneously vegetated surfaces |
| | B00 | Cropland |
| 4 (cropland) | B10 | Cereals |
| | B11 | Common wheat |
| | B12 | Durum wheat |
| | B13 | Barley |
| | B14 | Rye |
| | B15 | Oats |
| | B16 | Maize |
| | B19 | Other cereals |
| | B20 | Root crops |
| | B21 | Potatoes |
| | B22 | Sugar beet |
| | B23 | Other root crops |
| | B30 | Non-permanent industrial crops |
| | B31 | Sunflower |
| | B32 | Rape and turnip rape |
| | B33 | Soya |
| | B35 | Other fibre and oleaginous crops |
| | B37 | Other non-permanent industrial crops |
| | B40 | Dry pulses, vegetables and flowers |
| | B41 | Dry pulses |
| | B43 | Other fresh vegetables |
| | B44 | Floriculture and ornamental plants |
| | B45 | Strawberries |
| | B70 | Permanent crops: fruit trees |
| | B71 | Apple fruit |
| | B72 | Pear fruit |
| | B73 | Cherry fruit |
| | B75 | Other fruit trees and berries |
| | B80 | Other permanent crops |
| | B83 | Nurseries |
| | B84 | Permanent industrial crops |
| | BX1 | Arable land (only pi) |
| | BX2 | Permanent crops (only pi) |
| | C00 | Woodland |
| 5 (broadleaved woodland) | C10 | Broadleaved woodland |
| 6 (forest) | C20 | Coniferous woodland |
| | C21 | Spruce dominated coniferous woodland |
| | C22 | Pine dominated coniferous woodland |
| | C23 | Other coniferous woodland |
| | C30 | Mixed woodland |
| | C31 | Spruce dominated mixed woodland |
| | C32 | Pine dominated mixed woodland |
| | C33 | Other mixed woodland |
| 7 (shrubland) | D10 | Shrubland with sparse tree cover |
| | D20 | Shrubland without tree cover |
| | G00 | Water areas |
| 8 (waterbodies) | G11 | Inland freshwater bodies |
| | G12 | Inland salty water bodies |

| | | |
|---|---|---|
| | G21 | Inland fresh running water |
| | G22 | Inland salty running water |
| | G30 | Transitional water bodies |
| | H00 | Wetlands |
| 9 (wetlands) | H11 | Inland marshes |
| | H12 | Peatbogs |
| | H21 | Salt marshes |
| | H23 | Intertidal flats |

*Table D, E, F.. Model validation performance on RF, SVM and XGB respectively*

| **Random Forest** | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | | Baseline | | | Median of Neighbours | | | 10-Nearest Neighbours | | |
| Label | *n* | Precision | Recall | F1-score | Precision | Recall | F1-score | Precision | Recall | F1-score |
| 1 *(artificial land)* | 135 | 0.600 | 0.556 | 0.535 | 0.559 | 0.563 | 0.561 | 0.608 | 0.541 | 0.573 |
| 2 *(bare soil, rocks)* | 193 | 0.512 | 0.109 | 0.112 | 0.562 | 0.093 | 0.160 | 0.667 | 0.052 | 0.096 |
| 3 *(grasslands)* | 1247 | 0.713 | 0.924 | 0.731 | 0.714 | 0.925 | 0.806 | 0.674 | 0.939 | 0.785 |
| 4 *(crops)* | 462 | 0.743 | 0.745 | 0.578 | 0.732 | 0.764 | 0.748 | 0.698 | 0.665 | 0.681 |
| 5 *(broadleaved woodland)* | 234 | 0.672 | 0.526 | 0.025 | 0.665 | 0.500 | 0.571 | 0.697 | 0.453 | 0.549 |
| 6 *(forest)* | 135 | 0.673 | 0.163 | 0.272 | 0.699 | 0.533 | 0.605 | 0.647 | 0.489 | 0.557 |
| 7 *(shrubland)* | 131 | 0.636 | 0.008 | 0.015 | 0.800 | 0.183 | 0.298 | 0.889 | 0.061 | 0.114 |
| 8 *(waterbodies)* | 22 | 0.923 | 0.500 | 0.647 | 0.923 | 0.545 | 0.686 | 0.917 | 0.500 | 0.647 |
| 9 *(marshes)* | 29 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| Accuracy | | | | 0.705 | | | 0.706 | | | 0.677 |
| Macro Average | 2588 | 0.608 | 0.460 | 0.499 | 0.628 | 0.456 | 0.493 | 0.644 | 0.411 | 0.445 |
| Weighted Average | 2588 | 0.682 | 0.705 | 0.670 | 0.691 | 0.706 | 0.667 | 0.680 | 0.677 | 0.627 |

| **Support Vector Machine** | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | | Baseline | | | Median of Neighbours | | | 10-Nearest Neighbours | | |
| Label | *n* | Precision | Recall | F1-score | Precision | Recall | F1-score | Precision | Recall | F1-score |
| 1 *(artificial land)* | 135 | 0.550 | 0.615 | 0.580 | 0.579 | 0.681 | 0.626 | 0.550 | 0.444 | 0.492 |
| 2 *(bare soil, rocks)* | 193 | 0.474 | 0.093 | 0.156 | 0.435 | 0.140 | 0.212 | 0.378 | 0.088 | 0.143 |
| 3 *(grasslands)* | 1247 | 0.726 | 0.913 | 0.809 | 0.741 | 0.899 | 0.813 | 0.661 | 0.934 | 0.774 |
| 4 *(crops)* | 462 | 0.753 | 0.810 | 0.780 | 0.750 | 0.818 | 0.783 | 0.669 | 0.669 | 0.669 |
| 5 *(broadleaved woodland)* | 234 | 0.626 | 0.573 | 0.598 | 0.634 | 0.547 | 0.587 | 0.651 | 0.359 | 0.463 |
| 6 *(forest)* | 135 | 0.769 | 0.519 | 0.619 | 0.705 | 0.548 | 0.617 | 0.586 | 0.304 | 0.400 |
| 7 *(shrubland)* | 131 | 0.714 | 0.076 | 0.138 | 0.552 | 0.122 | 0.200 | 1.000 | 0.015 | 0.030 |
| 8 *(waterbodies)* | 22 | 0.714 | 0.455 | 0.556 | 0.667 | 0.455 | 0.541 | 0.778 | 0.318 | 0.452 |
| 9 *(marshes)* | 29 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| Accuracy | | | | 0.710 | | | 0.713 | | | 0.651 |
| Macro Average | 2588 | 0.592 | 0.450 | 0.471 | 0.562 | 0.468 | 0.486 | 0.586 | 0.348 | 0.380 |
| Weighted Average | 2588 | 0.687 | 0.710 | 0.669 | 0.671 | 0.713 | 0.680 | 0.642 | 0.651 | 0.597 |

| **xgboost** | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | | Baseline | | | Median of Neighbours | | | 10-Nearest Neighbours | | |
| Label | *n* | Precision | Recall | F1-score | Precision | Recall | F1-score | Precision | Recall | F1-score |
| 1 *(artificial land)* | 135 | 0.592 | 0.667 | 0.627 | 0.604 | 0.644 | 0.624 | 0.551 | 0.637 | 0.591 |
| 2 *(bare soil, rocks)* | 193 | 0.489 | 0.114 | 0.185 | 0.400 | 0.228 | 0.290 | 0.588 | 0.104 | 0.176 |
| 3 *(grasslands)* | 1247 | 0.738 | 0.911 | 0.816 | 0.763 | 0.896 | 0.824 | 0.727 | 0.915 | 0.810 |
| 4 *(crops)* | 462 | 0.736 | 0.766 | 0.751 | 0.765 | 0.766 | 0.765 | 0.712 | 0.755 | 0.733 |
| 5 *(broadleaved woodland)* | 234 | 0.642 | 0.530 | 0.581 | 0.610 | 0.547 | 0.577 | 0.640 | 0.509 | 0.567 |
| 6 *(forest)* | 135 | 0.695 | 0.541 | 0.608 | 0.718 | 0.548 | 0.622 | 0.667 | 0.533 | 0.593 |
| 7 *(shrubland)* | 131 | 0.614 | 0.267 | 0.372 | 0.611 | 0.336 | 0.433 | 0.733 | 0.168 | 0.273 |
| 8 *(waterbodies)* | 22 | 0.867 | 0.591 | 0.703 | 0.833 | 0.682 | 0.750 | 0.857 | 0.545 | 0.667 |
| 9 *(marshes)* | 29 | 0.000 | 0.000 | 0.000 | 0.200 | 0.034 | 0.059 | 0.000 | 0.000 | 0.000 |
| Accuracy | | | | 0.714 | | | 0.720 | | | 0.704 |
| Macro Average | 2588 | 0.597 | 0.487 | 0.516 | 0.612 | 0.520 | 0.549 | 0.608 | 0.463 | 0.490 |
| Weighted Average | 2588 | 0.687 | 0.714 | 0.683 | 0.699 | 0.720 | 0.702 | 0.687 | 0.704 | 0.667 |

```
import h5py
import pandas as pd
from datetime import datetime
from collections import defaultdict
import mapclassify
import matplotlib.pyplot as plt

file_path = "United_Kingdom.h5"

# Store counts by (lucas_point, month)
image_counts = defaultdict(int)

with h5py.File(file_path, 'r') as f:
    for key in f.keys():
        if key.startswith("Lucas_Point_"):
            attrs = f[key].attrs
            image_ids = attrs.get("Image_ID", [])
            for img_id in image_ids:
                if isinstance(img_id, bytes):
                    img_id = img_id.decode()
                try:
                    date = datetime.strptime(img_id[:8],
"%Y%m%d")
                    month = date.strftime("%Y-%m")
                    image_counts[(key, month)] += 1
                except:
                    continue

# Convert to DataFrame
records = [{"Lucas_Point": k[0], "Month": k[1], "Image_Count": v}
for k, v in image_counts.items()]
df = pd.DataFrame(records)

# Apply Jenks Natural Breaks
classifier = mapclassify.NaturalBreaks(y=df["Image_Count"], k=4)
df["Class"] = classifier.find_bin(df["Image_Count"])

# Reconstruct bin labels from classifier.bins
bin_edges = [df["Image_Count"].min()] + list(classifier.bins)
class_labels = [f"{int(bin_edges[i])}-{int(bin_edges[i+1])}" for
i in range(len(bin_edges)-1)]

# Convert "Month" to datetime and extract month name
df["Month_Name"] = pd.to_datetime(df["Month"]).dt.strftime("%b")

# Aggregate counts per month and class
agg = df.groupby(["Month_Name",
"Class"]).size().unstack(fill_value=0)
month_order = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
]
agg = agg.reindex(month_order)

# Plot
ax = agg.plot(
    kind='bar',
    stacked=True,
    figsize=(8, 4),  # Increased height from 5 → 6
    colormap="Set3"
)

plt.title("LUCAS Points by Number of Images per Month (Jenks
Natural Breaks)")
plt.ylabel("Number of Points", fontsize=12)
plt.xlabel("Month", fontsize=12)
plt.xticks(rotation=0)

# Legend placed just inside the top-right, avoiding overlap
plt.legend(
    title="Image Count Class",
    labels=class_labels,
    loc='upper center',
    bbox_to_anchor=(0.5, -0.15),  # Position below plot
    ncol=len(class_labels),
    frameon=True,
    fontsize=12,
    title_fontsize=11
)

plt.tight_layout(rect=[0, 0.05, 1, 1])  # Leave extra top margin
for legend
plt.show()

import h5py
import pandas as pd
from shapely.geometry import Point
import geopandas as gpd
import matplotlib.pyplot as plt
import contextily as ctx
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
from matplotlib import cm
import numpy as np

def extract_coords_and_labels(h5_file):
    """Extracts lon/lat coordinates and lc1 labels from the HDF5
file."""
    points = []

    with h5py.File(h5_file, 'r') as f:
        for key in f.keys():
            if key.startswith("Lucas_Point_"):
                attrs = f[key].attrs
                coords = attrs.get("Coordinates", None)
                lc1 = attrs.get("lc1", None)  # note lowercase
                if coords is not None and lc1 is not None:
                    try:
                        lon, lat = coords  # shapely wants (lon,
lat)
                        points.append({
                            "Longitude": lon,
                            "Latitude": lat,
                            "lc1": lc1.decode() if
isinstance(lc1, bytes) else str(lc1)
                        })
                    except Exception as e:
                        print(f"Skipping {key} due to error:
{e}")

    df = pd.DataFrame(points)
    return df

# Map Lc1 to labels and convert to GeoDataFrame
label_mapping = {
    'A10': 1, 'A11': 1, 'A12': 1, 'A13': 1, 'A20': 1, 'A21': 1,
'A30': 1,
    'A22': 2, 'F10': 2, 'F20': 2, 'F30': 2, 'F40': 2,
    'E10': 3, 'E20': 3, 'E30': 3, 'B50': 3, 'B51': 3, 'B52': 3,
'B53': 3,
    'B54': 3, 'B55': 3,
    'B10': 4, 'B11': 4, 'B12': 4, 'B13': 4, 'B14': 4, 'B15': 4,
'B16': 4,
    'B17': 4, 'B18': 4, 'B19': 4, 'B20': 4, 'B21': 4, 'B22': 4,
'B23': 4,
    'B30': 4, 'B31': 4, 'B32': 4, 'B33': 4, 'B34': 4, 'B35': 4,
'B36': 4,
    'B37': 4, 'B40': 4, 'B41': 4, 'B42': 4, 'B43': 4, 'B44': 4,
'B45': 4,
    'B70': 4, 'B71': 4, 'B72': 4, 'B73': 4, 'B74': 4, 'B75': 4,
'B76': 4,
    'B77': 4, 'B80': 4, 'B81': 4, 'B82': 4, 'B83': 4, 'B84': 4,
'BX1': 4,
    'BX2': 4, 'C10': 5, 'C20': 6, 'C21': 6, 'C22': 6, 'C23': 6,
'C30': 6,
    'C31': 6, 'C32': 6, 'C33': 6, 'CXX1': 6, 'CXX2': 6, 'CXX3':
6,
    'CXX4': 6, 'CXX5': 6, 'CXX6': 6, 'CXX7': 6, 'CXX8': 6,
'CXX9': 6,
    'CXXA': 6, 'CXXB': 6, 'CXXC': 6, 'CXXD': 6, 'CXXE': 6, 'D10':
7,
    'D20': 7, 'G10': 8, 'G11': 8, 'G12': 8, 'G20': 8, 'G21': 8,
'G22': 8,
    'G30': 8, 'G40': 8, 'G50': 8, 'H10': 9, 'H11': 9, 'H12': 9,
'H20': 9,
    'H21': 9, 'H22': 9, 'H23': 9
}

# Map numeric lc1 codes to labels
lc1_labels = {
    1: "1- Artificial land",
    2: "2- Bare soil, rocks",
    3: "3- Grassland",
    4: "4- Cropland",
    5: "5- Broadleaved Woodland",
    6: "6- Forest",
    7: "7- Shrubland",
    8: "8- Waterbodies",
    9: "9- Wetlands"
}

df = extract_coords_and_labels("United_Kingdom.h5")
df["lc1_num"] = df["lc1"].map(label_mapping)
df = df.dropna(subset=["lc1_num"])

# Convert to GeoDataFrame
geometry = [Point(xy) for xy in zip(df["Longitude"],
df["Latitude"])]
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs="EPSG:4326")
gdf = gdf.to_crs("EPSG:3857")  # Project to meters
gdf["lc1_label"] = gdf["lc1_num"].map(lc1_labels)

from mpl_toolkits.axes_grid1.inset_locator import inset_axes
from matplotlib import cm
import numpy as np

# 1. Count and simplify labels to class number only
lc_counts = gdf["lc1_label"].value_counts()
labels_sorted = sorted(lc_counts.index)  # ensures color
consistency
label_numbers = [label.split("-")[0].strip() for label in
labels_sorted]  # e.g., "1", "2", ...

values_sorted = [lc_counts[label] for label in labels_sorted]

# 2. Assign colors to match map
palette = cm.get_cmap("Set1", len(labels_sorted))
label_color_map = dict(zip(labels_sorted, [palette(i) for i in
range(len(labels_sorted))]))
colors = [label_color_map[label] for label in labels_sorted]

# 3. Main map
fig, ax = plt.subplots(figsize=(15, 8))
gdf.plot(
    ax=ax,
    column="lc1_label",
    categorical=True,
    legend=True,
    legend_kwds={'loc': 'center left', 'bbox_to_anchor': (0.52,
0.7), 'fontsize': 9},
```

```
    markersize=0.25,
    alpha=1.0,
    cmap="Set1"
)
ctx.add_basemap(ax,
source=ctx.providers.CartoDB.PositronNoLabels)
ax.set_xlim(-1200000, 1000000)
ax.set_ylim(6370000, 8630000)
ax.set_title("LUCAS Land Cover Distribution across the UK")
ax.axis('off')

# 4. Inset pie chart: move slightly lower & centered
inset_ax = inset_axes(ax, width="27%", height="27%", loc='center
right', bbox_to_anchor=(-0.02, 0.0, 0.85, 0.85),
bbox_transform=ax.transAxes)

# 5. Custom explode for thin slices
explode = [0.05 if v < max(values_sorted)*0.2 else 0 for v in
values_sorted]

# 6. Pie chart labels
wedges, texts, autotexts = inset_ax.pie(
    values_sorted,
    labels=None,
    autopct=lambda p: f"{int(round(p * sum(values_sorted) /
100)):,}" if p > 3 else '',  # only label larger slices
    startangle=90,
    counterclock=False,
    colors=colors,
    explode=explode,
    pctdistance=0.75,
    labeldistance=1.05,
    textprops={'fontsize': 9}
)

# Pie chart values
for i, wedge in enumerate(wedges):
    angle = (wedge.theta2 + wedge.theta1) / 2
    x = 0.6 * np.cos(np.deg2rad(angle))
    y = 0.6 * np.sin(np.deg2rad(angle))

    # Nudge label 8 and 9 slightly outward/down
    label = label_numbers[i]
    if label == '8':
        y += 0.15
    elif label == '9':
        y -= 0.15

    inset_ax.text(
        x, y,
        label,
        ha='center',
        va='center',
        fontsize=9,
        fontweight='bold'
    )

# Optional: move autotext (counts) outward a bit more
for autotext, wedge in zip(autotexts, wedges):
    if autotext.get_text():
        angle = (wedge.theta2 + wedge.theta1) / 2
        x = 1.3 * np.cos(np.deg2rad(angle))
        y = 1.3 * np.sin(np.deg2rad(angle))
        autotext.set_position((x, y))
        autotext.set_ha('center')
        autotext.set_va('center')
        autotext.set_fontsize(8)

# Add count labels manually for small slices like 8 and 9
for i, label in enumerate(label_numbers):
    if label in ['8', '9']:
        angle = (wedges[i].theta2 + wedges[i].theta1) / 2
        x = 1.5 * np.cos(np.deg2rad(angle))
        y = 1.5 * np.sin(np.deg2rad(angle))

        # Apply small vertical offsets to prevent overlap
        if label == '8':
            x -= 0.15
            y -= 0.1
        elif label == '9':
            x += 0.1

        count = values_sorted[i]

        inset_ax.text(
            x, y,
            f"{count:,}",
            ha='center',
            va='center',
            fontsize=8
        )

from matplotlib.patches import FancyArrow
from matplotlib_scalebar.scalebar import ScaleBar

# scalebar (using matplotlib-scalebar)
scalebar = ScaleBar(1, units='m', dimension='si-length',
location='upper left',
                    scale_loc='bottom', length_fraction=0.1,
width_fraction=0.01,
                    box_alpha=0.3, pad=0.4, border_pad=0.5)
ax.add_artist(scalebar)

# 2. north arrow
arrow_length = 100000  # in map units (meters); adjust to fit
scale
x, y = -1100000, 8450000  # position of the base of the arrow
(adjust for your map extent)

ax.annotate('N',
            xy=(x, y),
            xytext=(x, y - arrow_length),
            arrowprops=dict(facecolor='black', width=5,
headwidth=15),
            ha='center', va='center',
            fontsize=12, fontweight='bold')


plt.tight_layout()
plt.show()

import seaborn as sns

# Define a fixed color palette for lc1 classes using Seaborn
unique_classes = sorted(gdf["lc1_label"].unique())
palette = sns.color_palette("Set3", n_colors=len(unique_classes))
label_colors = dict(zip(unique_classes, palette))
# Count occurrences
lc_counts = gdf["lc1_label"].value_counts().sort_index()
total_count = lc_counts.sum()

# Match colors to class labels
bar_colors = [label_colors[label] for label in lc_counts.index]

# Plot bar chart
plt.figure(figsize=(8, 3))
ax = sns.barplot(x=lc_counts.values, y=lc_counts.index,
palette=bar_colors)
# Add percentage labels to the end of each bar
for i, (count, label) in enumerate(zip(lc_counts.values,
lc_counts.index)):
    percent = count / total_count * 100
    ax.text(
        count + total_count * 0.005,  # small offset to the right
        i,
        f"{percent:.1f}%",
        va='center',
        fontsize=9
    )
plt.xlabel("Number of LUCAS Points")
plt.ylabel("Land Cover Class")
plt.title("Distribution of Land Cover Classes in the UK")
plt.tight_layout()
plt.show()


import h5py
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
from sklearn.neighbors import BallTree
import numpy as np
import matplotlib.pyplot as plt

def extract_coords_and_labels(h5_file):
    """Extracts lon/lat coordinates and lc1 labels from the HDF5
file."""
    points = []

    with h5py.File(h5_file, 'r') as f:
        for key in f.keys():
            if key.startswith("Lucas_Point_"):
                attrs = f[key].attrs
                coords = attrs.get("Coordinates", None)
                lc1 = attrs.get("lc1", None)  # note lowercase

                if coords is not None and lc1 is not None:
                    try:
                        lon, lat = coords  # shapely wants (lon,
lat)
                        points.append({
                            "Longitude": lon,
                            "Latitude": lat,
                            "lc1": lc1.decode() if
isinstance(lc1, bytes) else str(lc1)
                        })
                    except Exception as e:
                        print(f"Skipping {key} due to error:
{e}")

    df = pd.DataFrame(points)
    return df

df = extract_coords_and_labels("United_Kingdom.h5")
print(f"✅ Extracted {len(df)} valid points")
print(df.head())

#  Convert to GeoDataFrame and project to metric CRS
geometry = [Point(xy) for xy in zip(df["Longitude"],
df["Latitude"])]
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs="EPSG:4326")
gdf = gdf.to_crs("EPSG:3857")  # metric (meters)

# Extract coordinates for BallTree
coords = np.vstack([gdf.geometry.x, gdf.geometry.y]).T
tree = BallTree(coords, metric='euclidean')

# Query distance to the nearest neighbor (excluding self)
# k=2 returns self (distance=0) + nearest neighbor
```

```
distances, _ = tree.query(coords, k=2)
nearest_distances = distances[:, 1]  # exclude self

# Histogram, zoomed in for readability
plt.figure(figsize=(8, 4))
plt.hist(nearest_distances, bins=100, color='skyblue',
edgecolor='black')

# Focus on distances below 2km
plt.xlim(0, 20000)

# Reference lines
plt.axvline(640, color='red', linestyle='--', label='640m (tile
size)')
plt.axvline(1280, color='orange', linestyle='--', label='1280m
(2x tile size buffer)')

plt.xlabel("Distance to Nearest Neighbor (meters)")
plt.ylabel("Number of Points")
plt.title("Nearest Neighbor Distance Distribution for LUCAS
Points")
plt.legend()
plt.tight_layout()
plt.show()

# Summary stats
print(f"Min distance: {nearest_distances.min():.2f} m")
print(f"Points < 640m apart: {(nearest_distances < 640).sum()}")
print(f"Points < 1280m apart: {(nearest_distances <
1280).sum()}")
print(f"Median NN distance: {np.median(nearest_distances):.2f}
m")
print(f"Mean NN distance: {nearest_distances.mean():.2f} m")


import h5py
import pandas as pd
from shapely.geometry import Point
import geopandas as gpd

def extract_coords_and_labels(h5_file):
    """Extracts lon/lat coordinates and lc1 labels from the HDF5
file."""
    points = []

    with h5py.File(h5_file, 'r') as f:
        for key in f.keys():
            if key.startswith("Lucas_Point_"):
                attrs = f[key].attrs
                coords = attrs.get("Coordinates", None)
                lc1 = attrs.get("lc1", None)  # note lowercase

                if coords is not None and lc1 is not None:
                    try:
                        lon, lat = coords  # shapely wants (lon,
lat)
                        points.append({
                            "Longitude": lon,
                            "Latitude": lat,
                            "lc1": lc1.decode() if
isinstance(lc1, bytes) else str(lc1)
                        })
                    except Exception as e:
                        print(f"Skipping {key} due to error:
{e}")

    df = pd.DataFrame(points)
    return df

df = extract_coords_and_labels("United_Kingdom.h5")
print(f"✅ Extracted {len(df)} valid points")
print(df.head())

# Map Lc1 to labels and convert to GeoDataFrame
label_mapping = {
    'A10': 1, 'A11': 1, 'A12': 1, 'A13': 1, 'A20': 1, 'A21': 1,
'A30': 1,
    'A22': 2, 'F10': 2, 'F20': 2, 'F30': 2, 'F40': 2,
    'E10': 3, 'E20': 3, 'E30': 3, 'B50': 3, 'B51': 3, 'B52': 3,
'B53': 3,
    'B54': 3, 'B55': 3,
    'B10': 4, 'B11': 4, 'B12': 4, 'B13': 4, 'B14': 4, 'B15': 4,
'B16': 4,
    'B17': 4, 'B18': 4, 'B19': 4, 'B20': 4, 'B21': 4, 'B22': 4,
'B23': 4,
    'B30': 4, 'B31': 4, 'B32': 4, 'B33': 4, 'B34': 4, 'B35': 4,
'B36': 4,
    'B37': 4, 'B40': 4, 'B41': 4, 'B42': 4, 'B43': 4, 'B44': 4,
'B45': 4,
    'B70': 4, 'B71': 4, 'B72': 4, 'B73': 4, 'B74': 4, 'B75': 4,
'B76': 4,
    'B77': 4, 'B80': 4, 'B81': 4, 'B82': 4, 'B83': 4, 'B84': 4,
'BX1': 4,
    'BX2': 4, 'C10': 5, 'C20': 6, 'C21': 6, 'C22': 6, 'C23': 6,
'C30': 6,
    'C31': 6, 'C32': 6, 'C33': 6, 'CXX1': 6, 'CXX2': 6, 'CXX3':
6,
    'CXX4': 6, 'CXX5': 6, 'CXX6': 6, 'CXX7': 6, 'CXX8': 6,
'CXX9': 6,
    'CXXA': 6, 'CXXB': 6, 'CXXC': 6, 'CXXD': 6, 'CXXE': 6, 'D10':
7,
    'D20': 7, 'G10': 8, 'G11': 8, 'G12': 8, 'G20': 8, 'G21': 8,
'G22': 8,
    'G30': 8, 'G40': 8, 'G50': 8, 'H10': 9, 'H11': 9, 'H12': 9,
'H20': 9,
    'H21': 9, 'H22': 9, 'H23': 9
}
df["lc1_num"] = df["lc1"].map(label_mapping)
df = df.dropna(subset=["lc1_num"])

# Convert to GeoDataFrame
geometry = [Point(xy) for xy in zip(df["Longitude"],
df["Latitude"])]
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs="EPSG:4326")
gdf = gdf.to_crs("EPSG:3857")  # Project to meters


# Calculate Moran's I across distance bands
from libpysal.weights import DistanceBand
from esda.moran import Moran
import matplotlib.pyplot as plt

distances_km = list(range(1, 11))
morans_I = []
p_values = []

for d in distances_km:
    print(f"Calculating Moran's I for {d} km...")
    w = DistanceBand.from_dataframe(gdf, threshold=d*1000,
silence_warnings=True, binary=True)
    mi = Moran(gdf["lc1_num"], w, two_tailed=False)
    morans_I.append(mi.I)
    p_values.append(mi.p_sim)

# Calculate Geary's C across distance bands
from esda.geary import Geary

geary_C = []
geary_p = []

for d in distances_km:
    print(f"Calculating Geary's C for {d} km...")
    w = DistanceBand.from_dataframe(gdf, threshold=d*1000,
silence_warnings=True, binary=True)
    gc = Geary(gdf["lc1_num"], w)
    geary_C.append(gc.C)
    geary_p.append(gc.p_sim)

plt.figure(figsize=(8, 4))

# Moran's I
plt.plot(distances_km, morans_I, marker='o', label="Moran's I",
color='royalblue')

# Geary's C
plt.plot(distances_km, geary_C, marker='s', label="Geary's C",
color='darkorange')

# Reference lines
plt.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.axhline(1, color='gray', linestyle=':', linewidth=1,
label="No autocorrelation (Geary's C = 1)")

plt.xlabel("Distance Threshold (km)")
plt.ylabel("Statistic Value")
plt.title("Moran's I vs. Geary's C Across Distance Bands")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Plotting p-values for Moran's I
plt.figure(figsize=(8, 5))
plt.plot(distances_km, p_values, marker='o', color='red',
label="p-value")
plt.axhline(0.05, color='gray', linestyle='--',
label="Significance threshold (0.05)")
plt.xlabel("Distance Threshold (km)")
plt.ylabel("p-value")
plt.title("Significance of Moran's I Across Distance Bands")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(distances_km, geary_p, marker='o', color='darkorange',
label="p-value")
plt.axhline(0.05, color='gray', linestyle='--',
label="Significance threshold (0.05)")
plt.xlabel("Distance Threshold (km)")
plt.ylabel("p-value")
plt.title("Significance of Geary's C Across Distance Bands")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# ---- Code from 03_ml-setup.ipynb ----

import h5py
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Crop image stack to desired size, preserving channels and time
def crop_center(img_b, cropx, cropy):
    c, t, y, x = img_b.shape
    startx = x // 2 - (cropx // 2)
    starty = y // 2 - (cropy // 2)
```

```
    return img_b[:, :, starty:starty + cropy, startx:startx +
cropx]

# Calculate median composite image for a given month
def calculate_monthly_composite(image, indices):
    monthly_stack = np.stack([image[idx] for idx in indices])
    return np.median(monthly_stack, axis=0)

from sklearn.model_selection import train_test_split

# Open the HDF5 file and get dataset keys
with h5py.File(file_path, "r") as h5_file:
    dataset_keys = list(h5_file.keys())  # Get all dataset names
(keys)

# Shuffle dataset keys for randomness
np.random.shuffle(dataset_keys)

# Split dataset keys into train (70%), test (15%), validation
(15%)
train_keys, temp_keys = train_test_split(dataset_keys,
test_size=0.3, random_state=42)
test_keys, val_keys = train_test_split(temp_keys, test_size=0.5,
random_state=42)

# Save split datasets into new HDF5 files (with attributes)
def save_hdf5(file_name, file_path, dataset_keys):
    with h5py.File(file_name, "w") as new_h5:
        with h5py.File(file_path, "r") as original_h5:
            for key in dataset_keys:
                # Copy dataset
                data = original_h5[key][()]
                dataset = new_h5.create_dataset(key, data=data,
compression="gzip")

                # Copy attributes
                for attr_name, attr_value in
original_h5[key].attrs.items():
                    dataset.attrs[attr_name] = attr_value  # Copy
attribute value

    print(f"Saved {len(dataset_keys)} datasets to {file_name}")


file_path = "United_Kingdom.h5"
# Save each split as a new HDF5 file
save_hdf5("uk_train.h5", file_path, train_keys)
save_hdf5("uk_test.h5", file_path, test_keys)
save_hdf5("uk_val.h5", file_path, val_keys)

import os
import h5py
import numpy as np
import pandas as pd

bands = ['B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B8A', 'B11',
'B12']
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

def crop_center(img_b, cropx, cropy):
    c, t, y, x = img_b.shape
    startx = x // 2 - (cropx // 2)
    starty = y // 2 - (cropy // 2)
    return img_b[:, :, starty:starty + cropy, startx:startx +
cropx]

def calculate_monthly_composite(image, indices):
    monthly_stack = np.stack([image[idx] for idx in indices])
    return np.median(monthly_stack, axis=0)

def process_split(h5_file_path, split_name):
    print(f"Processing {split_name}...")

    with h5py.File(h5_file_path, 'r') as f:
        keys = list(f.keys())
        monthly_data = {month: {band: [] for band in bands} for
month in months}
        Lc1, Lu1, Lucas_id = [], [], []

        for key in keys:
            im = f[key]
            mask = im['SCL'] < 9

            image = np.stack([np.where(mask == 1, im[band], 0)
for band in bands], axis=0)
            image = np.moveaxis(image, [0], [1])

            # Month-wise indices
            image_id_list = list(im.attrs['Image_ID'])
            month_indices = {
                month: [i for i, s in enumerate(image_id_list) if
f'2018{str(idx+1).zfill(2)}' in s]
                for idx, month in enumerate(months)
            }

            monthly_composites = []
            for month in months:
                if month_indices[month]:
                    composite =
calculate_monthly_composite(image, month_indices[month])
                else:
                    composite = np.zeros_like(image[0])
                monthly_composites.append(composite)

            stacked_image = np.stack(monthly_composites, axis=0)
            cropped_image = crop_center(stacked_image, 3, 3)

            for idx, month in enumerate(months):
                for band_idx, band in enumerate(bands):

monthly_data[month][band].append(cropped_image[idx][band_idx][1][
1])

            Lc1.append(im.attrs.get('lc1', 'Unknown'))
            Lu1.append(im.attrs.get('lu1', 'Unknown'))
            Lucas_id.append(key[12:])

    # Save to CSV
    data = {'Lucas_ID': Lucas_id}
    for month in months:
        for band in bands:
            data[f'{month}_{band}'] = monthly_data[month][band]
    data['Lc1'] = Lc1
    data['Lu1'] = Lu1

    df = pd.DataFrame(data)
    output_path = os.path.join(f'uk_monthly_{split_name}.csv')
    df.to_csv(output_path, index=False)
    print(f"Saved CSV to {output_path}")

# Run for all splits
if __name__ == "__main__":
    for split in ['train', 'val', 'test']:
        h5_file_path = os.path.join(f'uk_{split}.h5')
        process_split(h5_file_path, split)

# Normalization of data

import os
import numpy as np
import pandas as pd


def normalise_data(split):
    file_path = os.path.join(f'uk_monthly_{split}.csv')
    df = pd.read_csv(file_path)

    bands = [col for col in df.columns if '_' in col and col not
in ['Lc1', 'Lu1', 'Lucas_ID']]
    df = df.dropna(subset=['Lc1'])

    for band in bands:
        q_low, q_hi = df[band].quantile(0.01),
df[band].quantile(0.999)
        df[band] = (df[band] - q_low) / (q_hi - q_low + np.exp(-
12))
        df[band] = df[band].clip(0, 1)

    output_path = os.path.join(f'uk_monthly_{split}_norm.csv')
    df.to_csv(output_path, index=False)
    print(f"Saved normalized CSV to {output_path}")

# Run for each split
if __name__ == "__main__":
    for split in ['train', 'val', 'test']:
        normalise_data(split)

# Creating the adjacency matrix for the UK dataset using the 10
nearest neighbors
import h5py
import pandas as pd
import numpy as np
from scipy.spatial import cKDTree

def extract_coordinates(h5_file):
    """Extracts Lucas point coordinates from the HDF5 file."""
    coords = []
    ids = []

    with h5py.File(h5_file, 'r') as f:
        for key in f.keys():
            if key.startswith("Lucas_Point_"):
                point_id = key.split("_")[-1]  # Extract ID
                lat, lon = f[key].attrs["Coordinates"]  # (lat,
lon)
                coords.append([lat, lon])
                ids.append(point_id)

    return np.array(coords), ids

def create_nearest_neighbors(coords, ids, k=10):
    """Finds the k nearest neighbors (excluding self) and their
Euclidean distances using KDTree."""
    tree = cKDTree(coords)
    adjacency_list = {id_: [] for id_ in ids}

    # Query k+1 because the nearest neighbor is always the point
itself
    distances, indices = tree.query(coords, k=k+1)

    for i, lucas_id in enumerate(ids):
        neighbors = indices[i][1:]  # Exclude self (first index
is the point itself)
        dists = distances[i][1:]  # Exclude self distance (0)
        adjacency_list[lucas_id] = [(ids[j], dists[idx]) for idx,
j in enumerate(neighbors)]

    return adjacency_list
```

```
def save_nearest_neighbors(adjacency_list, output_file):
    """Save nearest neighbors list to a CSV file."""
    adjacency_data = []
    for lucas_id, neighbors in adjacency_list.items():
        for neighbor_id, distance in neighbors:
            adjacency_data.append([lucas_id, neighbor_id,
distance])

    adjacency_df = pd.DataFrame(adjacency_data,
columns=['Lucas_ID', 'Neighbor_ID', 'Distance'])
    adjacency_df.to_csv(output_file, index=False)
    print(f"Nearest neighbors saved to {output_file}")

# File path
h5_file_path = "United_Kingdom.h5"
# Extract coordinates
coords, ids = extract_coordinates(h5_file_path)
# Compute nearest neighbors
adjacency_list = create_nearest_neighbors(coords, ids, k=10)
# Save results
save_nearest_neighbors(adjacency_list, "uk_10nn_matrix.csv")

# Adapting the train, test, val csv to include the 10-NN band
values
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# File paths
train_file = "uk_monthly_train.csv"
test_file = "uk_monthly_test.csv"
val_file = "uk_monthly_val.csv"
adjacency_file = "uk_10nn_matrix.csv"

# Load datasets
train_df = pd.read_csv(train_file)
test_df = pd.read_csv(test_file)
val_df = pd.read_csv(val_file)
adjacency_df = pd.read_csv(adjacency_file)

# Merge all datasets
all_data = pd.concat([train_df, test_df, val_df],
ignore_index=True)

# Extract band columns
band_columns = [col for col in train_df.columns if
col.startswith(('Jan_', 'Feb_', 'Mar_', 'Apr_', 'May_', 'Jun_',

'Jul_', 'Aug_', 'Sep_', 'Oct_', 'Nov_', 'Dec_'))]

# Create a dictionary mapping Lucas_ID to its band values
data_dict =
all_data.set_index("Lucas_ID")[band_columns].to_dict(orient="inde
x")

# Create adjacency dictionary with distances
adjacency_df = adjacency_df.sort_values(by=["Lucas_ID",
"Distance"])  # Ensure nearest are in order
adj_dict =
adjacency_df.groupby("Lucas_ID")["Neighbor_ID"].apply(list).to_di
ct()

def append_neighbor_bands(df, k=10):
    """Appends band values from the k nearest neighbors."""
    neighbor_bands = {f"{col}_n{n+1}": np.zeros(len(df)) for col
in band_columns for n in range(k)}

    for i, lucas_id in enumerate(df["Lucas_ID"]):
        neighbors = adj_dict.get(lucas_id, [])[:k]  # Get up to k
neighbors

        for n, neighbor_id in enumerate(neighbors):
            if neighbor_id in data_dict:
                for col in band_columns:
                    neighbor_bands[f"{col}_n{n+1}"][i] =
data_dict[neighbor_id][col]  # Assign neighbor band value

    # Append new columns
    for col in band_columns:
        for n in range(k):
            df[f"{col}_n{n+1}"] = neighbor_bands[f"{col}_n{n+1}"]

    return df

# Append nearest neighbor bands for all datasets
train_df = append_neighbor_bands(train_df, k=10)
test_df = append_neighbor_bands(test_df, k=10)
val_df = append_neighbor_bands(val_df, k=10)

# Merge again to ensure normalization is applied consistently
all_data = pd.concat([train_df, test_df, val_df],
ignore_index=True)

# Normalize using Min-Max Scaling
scaler = MinMaxScaler()
all_band_cols = band_columns + [f"{col}_n{n+1}" for col in
band_columns for n in range(10)]
all_data[all_band_cols] =
scaler.fit_transform(all_data[all_band_cols])

# Drop 'Lu1' column if it exists
if 'Lu1' in all_data.columns:
    all_data = all_data.drop(columns=['Lu1'])

# Split datasets back
train_df = all_data.iloc[:len(train_df)]
test_df = all_data.iloc[len(train_df):len(train_df) +
len(test_df)]
val_df = all_data.iloc[len(train_df) + len(test_df):]

# Save updated files
train_df.to_csv("uk_monthly_train_10nn_norm.csv", index=False)
test_df.to_csv("uk_monthly_test_10nn_norm.csv", index=False)
val_df.to_csv("uk_monthly_val_10nn_norm.csv", index=False)

print("Updated files saved with normalized 10NN band columns.")

# Creating an adjacency matrix with LUCAS points within 5.0km
import h5py
import pandas as pd
import numpy as np
from scipy.spatial import cKDTree

def extract_coordinates(h5_file):
    """Extracts Lucas point coordinates from the HDF5 file."""
    coords = []
    ids = []

    with h5py.File(h5_file, 'r') as f:
        for key in f.keys():
            if key.startswith("Lucas_Point_"):
                point_id = key.split("_")[-1]  # Extract ID
                lat, lon = f[key].attrs["Coordinates"]  # (lat,
lon)
                coords.append([lat, lon])
                ids.append(point_id)

    return np.array(coords), ids

def create_adjacency_matrix(coords, ids, radius=5.0):
    """Create adjacency list of Lucas_IDs within the specified
radius using KDTree."""
    # cKDTree for fast nn search - far more efficient than brute
force loop!!
    tree = cKDTree(coords)
    adjacency_list = {id_: [] for id_ in ids}

    for i, lucas_id in enumerate(ids):
        # Query neighbors within 5km (Earth radius ~6371km, so
convert to degrees)
        idx = tree.query_ball_point(coords[i], radius / 6371 *
(180 / np.pi))
        adjacency_list[lucas_id] = [ids[j] for j in idx if j !=
i]  # Exclude self

    return adjacency_list

def save_adjacency_matrix(adjacency_list, output_file):
    """Save adjacency list to a CSV file."""
    adjacency_data = []
    for lucas_id, neighbors in adjacency_list.items():
        for neighbor in neighbors:
            adjacency_data.append([lucas_id, neighbor])

    adjacency_df = pd.DataFrame(adjacency_data,
columns=['Lucas_ID', 'Neighbor_ID'])
    adjacency_df.to_csv(output_file, index=False)
    print(f"Adjacency matrix saved to {output_file}")


h5_file_path = "United_Kingdom.h5"
# 1: Extract coordinates
coords, ids = extract_coordinates(h5_file_path)
# 2: Compute adjacency matrix
adjacency_list = create_adjacency_matrix(coords, ids, radius=5.0)
# 3: Save matrix
save_adjacency_matrix(adjacency_list, "uk_adj_matrix.csv")


# Create train, test, val datasets with adjacent median values
and normalized band values
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# File paths
train_file = "uk_monthly_train.csv"
test_file = "uk_monthly_test.csv"
val_file = "uk_monthly_val.csv"
adjacency_file = "uk_adj_matrix.csv"

# Load datasets
train_df = pd.read_csv(train_file)
test_df = pd.read_csv(test_file)
val_df = pd.read_csv(val_file)
adjacency_df = pd.read_csv(adjacency_file)

# Merge all datasets
all_data = pd.concat([train_df, test_df, val_df],
ignore_index=True)

# Extract band columns
band_columns = [col for col in train_df.columns if
col.startswith(('Jan_', 'Feb_', 'Mar_', 'Apr_', 'May_', 'Jun_',

'Jul_', 'Aug_', 'Sep_', 'Oct_', 'Nov_', 'Dec_'))]

# Create a dictionary mapping Lucas_ID to its row of band values
```

```
data_dict =
all_data.set_index("Lucas_ID")[band_columns].to_dict(orient="inde
x")

# Create adjacency dictionary for fast lookup
adj_dict =
adjacency_df.groupby("Lucas_ID")["Neighbor_ID"].apply(list).to_di
ct()

def compute_adjacent_medians(df):
    """Compute median band values of adjacent points using
efficient lookups."""
    adj_values = {col: np.zeros(len(df)) for col in band_columns}

    for i, lucas_id in enumerate(df["Lucas_ID"]):
        neighbors = adj_dict.get(lucas_id, [])

        if neighbors:
            neighbor_values =
np.array([list(data_dict[n].values()) for n in neighbors if n in
data_dict])

            if neighbor_values.size > 0:
                median_vals = np.median(neighbor_values, axis=0)
            else:
                median_vals = np.zeros(len(band_columns))  #
Default if no valid neighbors
        else:
            median_vals = np.zeros(len(band_columns))  # Default
if no neighbors

        for j, col in enumerate(band_columns):
            adj_values[col][i] = median_vals[j]

    # Append new columns
    for col in band_columns:
        df[f"{col}_adj"] = adj_values[col]

    return df

# Compute adjacent medians for all datasets
train_df = compute_adjacent_medians(train_df)
test_df = compute_adjacent_medians(test_df)
val_df = compute_adjacent_medians(val_df)

# Merge again to ensure normalization is applied consistently
all_data = pd.concat([train_df, test_df, val_df],
ignore_index=True)

# Normalize using Min-Max Scaling
scaler = MinMaxScaler()
all_data[band_columns + [f"{col}_adj" for col in band_columns]] =
scaler.fit_transform(
    all_data[band_columns + [f"{col}_adj" for col in
band_columns]]
)

# Drop 'Lu1' column if it exists
if 'Lu1' in all_data.columns:
    all_data = all_data.drop(columns=['Lu1'])

# Split datasets back
train_df = all_data.iloc[:len(train_df)]
test_df = all_data.iloc[len(train_df):len(train_df) +
len(test_df)]
val_df = all_data.iloc[len(train_df) + len(test_df):]

# Save normalized files
train_df.to_csv("uk_monthly_train_adj_norm.csv", index=False)
test_df.to_csv("uk_monthly_test_adj_norm.csv", index=False)
val_df.to_csv("uk_monthly_val_adj_norm.csv", index=False)

print("Updated files saved with normalized adjacent median
columns.")

# ---- Code from 04_ml-adj.ipynb ----

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

def load_data(file_path):
    df = pd.read_csv(file_path)
    bands = [col for col in df.columns if
any(col.startswith(f"{month}_B") for month in
            ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul",
"Aug", "Sep", "Oct", "Nov", "Dec"])]

    X = df[bands].values

    label_mapping = {
        'A10': 1, 'A11': 1, 'A12': 1, 'A13': 1, 'A20': 1, 'A21':
1, 'A30': 1,
        'A22': 2, 'F10': 2, 'F20': 2, 'F30': 2, 'F40': 2,
        'E10': 3, 'E20': 3, 'E30': 3, 'B50': 3, 'B51': 3, 'B52':
3, 'B53': 3,
        'B54': 3, 'B55': 3,
        'B10': 4, 'B11': 4, 'B12': 4, 'B13': 4, 'B14': 4, 'B15':
4, 'B16': 4,
        'B17': 4, 'B18': 4, 'B19': 4, 'B20': 4, 'B21': 4, 'B22':
4, 'B23': 4,
        'B30': 4, 'B31': 4, 'B32': 4, 'B33': 4, 'B34': 4, 'B35':
4, 'B36': 4,
        'B37': 4, 'B40': 4, 'B41': 4, 'B42': 4, 'B43': 4, 'B44':
4, 'B45': 4,
        'B70': 4, 'B71': 4, 'B72': 4, 'B73': 4, 'B74': 4, 'B75':
4, 'B76': 4,
        'B77': 4, 'B80': 4, 'B81': 4, 'B82': 4, 'B83': 4, 'B84':
4, 'BX1': 4,
        'BX2': 4, 'C10': 5, 'C20': 6, 'C21': 6, 'C22': 6, 'C23':
6, 'C30': 6,
        'C31': 6, 'C32': 6, 'C33': 6, 'CXX1': 6, 'CXX2': 6,
'CXX3': 6,
        'CXX4': 6, 'CXX5': 6, 'CXX6': 6, 'CXX7': 6, 'CXX8': 6,
'CXX9': 6,
        'CXXA': 6, 'CXXB': 6, 'CXXC': 6, 'CXXD': 6, 'CXXE': 6,
'D10': 7,
        'D20': 7, 'G10': 8, 'G11': 8, 'G12': 8, 'G20': 8, 'G21':
8, 'G22': 8,
        'G30': 8, 'G40': 8, 'G50': 8, 'H10': 9, 'H11': 9, 'H12':
9, 'H20': 9,
        'H21': 9, 'H22': 9, 'H23': 9
    }
    y = df['Lc1'].map(label_mapping).values
    return X, y

# Load train and test data
X_train, y_train = load_data('uk_monthly_train_adj_norm.csv')
X_test, y_test = load_data('uk_monthly_test_adj_norm.csv')

# GridSearchCV Hyperparameter tuning for Random Forest Classifier
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid_search =
GridSearchCV(RandomForestClassifier(random_state=42), param_grid,
cv=3, scoring="accuracy", n_jobs=-1)
grid_search.fit(X_train, y_train)

best_rf = grid_search.best_estimator_
print("Best parameters:", grid_search.best_params_)

# Bayesian Optimization with Optuna Hyperparameter tuning for
Random Forest Classifier
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.metrics import f1_score

def objective(trial):
    model = RandomForestClassifier(
        n_estimators=trial.suggest_int("n_estimators", 100, 500),
        max_depth=trial.suggest_int("max_depth", 5, 50),
        min_samples_split=trial.suggest_int("min_samples_split",
2, 10),
        min_samples_leaf=trial.suggest_int("min_samples_leaf", 1,
5),
        random_state=42,
        n_jobs=-1
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    return f1_score(y_test, y_pred, average='macro')

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)

print("Best parameters:", study.best_params)

optuna_rf = RandomForestClassifier(
    n_estimators=439,
    max_depth=34,
    min_samples_split=6,
    min_samples_leaf=1,
    random_state=42
)
optuna_rf.fit(X_train, y_train)

import pandas as pd
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

# Load Data (Assuming X_train, X_test, y_train, y_test are
available)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define hyperparameter grid
param_grid = {
    'C': [0.1, 1, 10, 100],  # Regularization parameter
    'gamma': ['scale', 'auto', 0.01, 0.1, 1],  # Kernel
coefficient
    'kernel': ['rbf', 'poly'],  # Try different kernels
    'class_weight': ['balanced', None]  # Address class imbalance
}

# Initialize SVM and GridSearch
svm = SVC()
grid_search = GridSearchCV(svm, param_grid, cv=5,
scoring='accuracy', n_jobs=-1, verbose=2)
```

```
grid_search.fit(X_train_scaled, y_train)

# Best parameters
print(f"Best Parameters: {grid_search.best_params_}")

# Predict using the best model
best_svm = grid_search.best_estimator_
y_pred_svm = best_svm.predict(X_test_scaled)


# Evaluation
accuracy = accuracy_score(y_test, y_pred_svm)
print(f"SVM Accuracy: {accuracy:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_svm))

# Bayesian Optimization with Optuna Hyperparameter tuning for SVM
from sklearn.svm import SVC
from sklearn.metrics import f1_score
def objective(trial):
    C = trial.suggest_loguniform("C", 1e-2, 1e2)
    gamma = trial.suggest_loguniform("gamma", 1e-4, 1e0)
    kernel = trial.suggest_categorical("kernel", ["rbf", "poly"])
    class_weight = trial.suggest_categorical("class_weight",
[None, "balanced"])

    model = SVC(C=C, gamma=gamma, kernel=kernel,
class_weight=class_weight, random_state=42)
    model.fit(X_train_scaled, y_train)

    y_pred = model.predict(X_test_scaled)  # or use
cross_val_score
    return f1_score(y_test, y_pred, average='macro')

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

best_svm_optuna = SVC(
    C=22.90420869178104,
    gamma=0.00041822818832112116,
    kernel='rbf',
    class_weight=None,
    random_state=42
)
best_svm_optuna.fit(X_train_scaled, y_train)

import xgboost as xgb
import numpy as np
from sklearn.model_selection import GridSearchCV

# Remap label 9 to 0 (xgbclassifier requires labels to start from
0, not 1)
y_train = np.where(y_train == 9, 0, y_train)
y_test = np.where(y_test == 9, 0, y_test)

# Define XGBoost classifier
xgb_model = xgb.XGBClassifier(objective="multi:softmax",
num_class=9, eval_metric="mlogloss", use_label_encoder=False)

# Define the parameter grid
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.01, 0.1, 0.2],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0]
}

# Set up GridSearchCV
grid_search_xgb = GridSearchCV(estimator=xgb_model,
param_grid=param_grid,
                               scoring="accuracy", cv=3,
verbose=2, n_jobs=-1)

# Run Grid Search
grid_search_xgb.fit(X_train, y_train)

# Get the best parameters
print("Best Parameters:", grid_search_xgb.best_params_)

# Train best model
best_xgb = grid_search_xgb.best_estimator_

# Make predictions
y_pred_xgb = best_xgb.predict(X_test)

# Convert predicted label 0 back to 9 for consistency
y_pred_xgb = np.where(y_pred_xgb == 0, 9, y_pred_xgb)
# Ensure y_test is also converted back for proper evaluation
y_test = np.where(y_test == 0, 9, y_test)

import optuna
from sklearn.metrics import f1_score
import xgboost as xgb

def objective(trial):
    params = {
        "objective": "multi:softmax",
        "num_class": 9,
        "eval_metric": "mlogloss",
        "use_label_encoder": False,
        "n_estimators": trial.suggest_int("n_estimators", 100,
500),
        "max_depth": trial.suggest_int("max_depth", 3, 15),
        "learning_rate": trial.suggest_float("learning_rate",
0.01, 0.3),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree":
trial.suggest_float("colsample_bytree", 0.6, 1.0),
        "gamma": trial.suggest_float("gamma", 0, 5),
        "min_child_weight": trial.suggest_int("min_child_weight",
1, 10),
        "random_state": 42,
        "n_jobs": -1
    }

    model = xgb.XGBClassifier(**params)

    # Remap label 9 to 0 for XGBoost
    y_train_remapped = np.where(y_train == 9, 0, y_train)
    y_test_remapped = np.where(y_test == 9, 0, y_test)

    model.fit(X_train, y_train_remapped)
    preds = model.predict(X_test)

    # Convert predictions back to original labels for evaluation
    preds = np.where(preds == 0, 9, preds)
    y_test_true = np.where(y_test_remapped == 0, 9,
y_test_remapped)

    return f1_score(y_test_true, preds, average='macro')

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

best_xgb = xgb.XGBClassifier(
    objective="multi:softmax",
    num_class=9,
    eval_metric="mlogloss",
    use_label_encoder=False,
    n_estimators=438,
    max_depth=3,
    learning_rate=0.2155121784596623,
    subsample=0.7616832743549153,
    colsample_bytree=0.8483548247399473,
    gamma=0.9334204872821765,
    min_child_weight=8,
    random_state=42,
    n_jobs=-1
)

# Train on remapped labels
y_train_remap = np.where(y_train == 9, 0, y_train)
best_xgb.fit(X_train, y_train_remap)

def load_data(file_path):
    df = pd.read_csv(file_path)

    # Extract bands for features
    bands = [col for col in df.columns if
any(col.startswith(f"{month}_B") for month in
          ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul",
"Aug", "Sep", "Oct", "Nov", "Dec"])]

    # Label mapping
    label_mapping = {
        'A10': 1, 'A11': 1, 'A12': 1, 'A13': 1, 'A20': 1, 'A21':
1, 'A30': 1,
        'A22': 2, 'F10': 2, 'F20': 2, 'F30': 2, 'F40': 2,
        'E10': 3, 'E20': 3, 'E30': 3, 'B50': 3, 'B51': 3, 'B52':
3, 'B53': 3,
        'B54': 3, 'B55': 3,
        'B10': 4, 'B11': 4, 'B12': 4, 'B13': 4, 'B14': 4, 'B15':
4, 'B16': 4,
        'B17': 4, 'B18': 4, 'B19': 4, 'B20': 4, 'B21': 4, 'B22':
4, 'B23': 4,
        'B30': 4, 'B31': 4, 'B32': 4, 'B33': 4, 'B34': 4, 'B35':
4, 'B36': 4,
        'B37': 4, 'B40': 4, 'B41': 4, 'B42': 4, 'B43': 4, 'B44':
4, 'B45': 4,
        'B70': 4, 'B71': 4, 'B72': 4, 'B73': 4, 'B74': 4, 'B75':
4, 'B76': 4,
        'B77': 4, 'B80': 4, 'B81': 4, 'B82': 4, 'B83': 4, 'B84':
4, 'BX1': 4,
        'BX2': 4, 'C10': 5, 'C20': 6, 'C21': 6, 'C22': 6, 'C23':
6, 'C30': 6,
        'C31': 6, 'C32': 6, 'C33': 6, 'CXX1': 6, 'CXX2': 6,
'CXX3': 6,
        'CXX4': 6, 'CXX5': 6, 'CXX6': 6, 'CXX7': 6, 'CXX8': 6,
'CXX9': 6,
        'CXXA': 6, 'CXXB': 6, 'CXXC': 6, 'CXXD': 6, 'CXXE': 6,
'D10': 7,
        'D20': 7, 'G10': 8, 'G11': 8, 'G12': 8, 'G20': 8, 'G21':
8, 'G22': 8,
        'G30': 8, 'G40': 8, 'G50': 8, 'H10': 9, 'H11': 9, 'H12':
9, 'H20': 9,
        'H21': 9, 'H22': 9, 'H23': 9
    }

    X = df[bands].values
    y = df['Lc1'].map(label_mapping).values
    lucas_ids = df['Lucas_ID']  # Extract Lucas_ID column

    return X, y, lucas_ids

# Load validation data
X_val, y_val, lucas_ids =
load_data('uk_monthly_val_adj_norm.csv')
```

```
from sklearn.metrics import classification_report,
confusion_matrix

def evaluate_model(y_true, y_pred, model_name):
    print(f"\n🔍 {model_name} Evaluation:\n")
    print(classification_report(y_true, y_pred, digits=3))
    print("Confusion Matrix:")
    print(confusion_matrix(y_true, y_pred))

rf_val_pred = optuna_rf.predict(X_val)

evaluate_model(y_val, rf_val_pred, "Optuna Random Forest")

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)
X_val_scaled = scaler.transform(X_val)

svm_val_pred = best_svm_optuna.predict(X_val_scaled)
evaluate_model(y_val, svm_val_pred, "Optuna SVM")

# Predict and remap
y_val_remap = np.where(y_val == 9, 0, y_val)
xgb_val_preds = best_xgb.predict(X_val)
xgb_val_preds = np.where(xgb_val_preds == 0, 9, xgb_val_preds)
y_val_orig = np.where(y_val_remap == 0, 9, y_val_remap)

evaluate_model(y_val_orig, xgb_val_preds, "XGBoost")

# Save predictions to CSV
predictions_df = pd.DataFrame({
    "Lucas_ID": lucas_ids,
    "True_Label": y_val,
    "RF_Prediction": rf_val_pred,
    "SVM_Prediction": svm_val_pred,
    "XGB_Prediction": xgb_val_preds
})

predictions_df.to_csv("adj_predictions.csv", index=False)

print("✅ Predictions saved to adj_predictions.csv")

# result.importances_mean gives the average drop in accuracy per
feature
# result.importances_std gives uncertainty across permutations
import matplotlib.pyplot as plt
import numpy as np
from sklearn.inspection import permutation_importance

# Permutation Importance
# Visualise top features
result = permutation_importance(optuna_rf, X_val, y_val,
n_repeats=10, random_state=42, n_jobs=-1)

df = pd.read_csv("uk_monthly_train_adj_norm.csv")
feature_names = df.columns.tolist()  # Get all feature columns
# Extract Sentinel-2 bands (including neighbor features like _n1
to _n10)
band_features = [col for col in feature_names if
any(col.startswith(f"{month}_B") for month in
                ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
"Jul", "Aug", "Sep", "Oct", "Nov", "Dec"])]

month_colors = {
    "Jan": "#5da0d7",
    "Feb": "#00b3e1",
    "Mar": "#65bda5",
    "Apr": "#87bf54",
    "May": "#cedale",
    "Jun": "#fee327",
    "Jul": "#fdca54",
    "Aug": "#ff9160",
    "Sep": "#fe81b1",
    "Oct": "#aa76c6",
    "Nov": "#927db6",
    "Dec": "#7473cd",
}

feature_names = np.array(band_features)
sorted_idx = result.importances_mean.argsort()[::-1][:30]  # Top
30
top_feature_names = feature_names[sorted_idx]

# Extract month names from feature names
month_labels = [name.split('_')[0] for name in top_feature_names]
# Create color list based on month
bar_colors = [month_colors.get(month, "#000000") for month in
month_labels]

plt.figure(figsize=(9, 5.2))
plt.barh(np.array(feature_names)[sorted_idx],
result.importances_mean[sorted_idx], color=bar_colors)
plt.xlabel("Mean Accuracy Decrease")
plt.title("Permutation Feature Importance (Top 30)")
plt.gca().invert_yaxis()

handles = [plt.Rectangle((0, 0), 1, 1, color=color) for color in
month_colors.values()]
labels = list(month_colors.keys())
plt.legend(handles, labels, title="Months", loc='lower right',
prop={'size': 13})

plt.tight_layout()
plt.show()

# Plot Confusion Matrices
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix,
classification_report

labels = sorted(np.unique(y_val))  # Should be [1, 2, ..., 9]

# RF
rf_preds = optuna_rf.predict(X_val)
cm_rf = confusion_matrix(y_val, rf_preds)

# SVM
svm_preds = best_svm_optuna.predict(X_val_scaled)
svm_preds = svm_preds.astype(int)
svm_preds = np.where(svm_preds == 0, 9, svm_preds)

cm_svm = confusion_matrix(y_val, svm_preds)

# XGB
y_val_remap = np.where(y_val == 9, 0, y_val)
xgb_preds = best_xgb.predict(X_val)
xgb_preds = np.where(xgb_preds == 0, 9, xgb_preds)
y_val_orig = np.where(y_val_remap == 0, 9, y_val_remap)
cm_xgb = confusion_matrix(y_val_orig, xgb_preds)

# --- Plotting ---
fig, axes = plt.subplots(1, 3, figsize=(20, 5))
model_names = ["Random Forest", "SVM", "XGBoost"]
conf_matrices = [cm_rf, cm_svm, cm_xgb]

for ax, cm, title in zip(axes, conf_matrices, model_names):
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=labels, yticklabels=labels, ax=ax,
annot_kws={"size": 13})
    ax.set_title(title, fontsize=16)
    ax.set_xlabel("Predicted", fontsize=13)
    ax.set_ylabel("True", fontsize=13)

plt.tight_layout()
plt.suptitle("Confusion Matrices for Optuna-Tuned Models",
fontsize=16, y=1.05)
plt.show()

import h5py
import pandas as pd
import matplotlib.pyplot as plt
import geopandas as gpd
from shapely.geometry import Point

# Load validation dataset
val_file = "uk_monthly_val_adj_norm.csv"
def load_validation_data(file_path):
    df = pd.read_csv(file_path)
    return df[['Lucas_ID', 'Lc1']]

df_val = load_validation_data(val_file)

# Load coordinates from h5 dataset
h5_file_path = "uk_val.h5"
def load_coordinates(file_path):
    points = []
    with h5py.File(file_path, "r") as h5_file:
        for key in h5_file.keys():
            coords = h5_file[key].attrs.get("Coordinates", [None,
None])
            if coords is not None:
                numeric_key = int(key.split("_")[-1])
                points.append({"Lucas_Point": numeric_key, "lat":
coords[1], "lon": coords[0]})
    df = pd.DataFrame(points)
    df["Lucas_Point"] = df["Lucas_Point"].astype(int)  # Convert
to int for merging
    return df

df_coords = load_coordinates(h5_file_path)

# Merge datasets
merged_df = df_val.merge(df_coords, left_on="Lucas_ID",
right_on="Lucas_Point", how="inner")

# Load model predictions from combined CSV
predictions_file = "adj_predictions.csv"
df_preds = pd.read_csv(predictions_file)

# Merge predictions
merged_df = merged_df.merge(df_preds, on="Lucas_ID", how="inner")

# Define models
models = ["RF", "SVM", "XGB"]

# Load UK boundary shapefile
uk_boundary_file = "Countries_December_2019_FCB_UK.shp"
uk_boundary =
gpd.read_file(uk_boundary_file).to_crs("EPSG:27700")

import matplotlib.patches as mpatches
from matplotlib_scalebar.scalebar import ScaleBar

# Mapping of label IDs to class names
label_names = {
    2: "Bare soil, rocks",
    7: "Shrubland"
```

```
}

# Function to assign colors for each label-specific map
def assign_colors_label(row, model, label):
    try:
        true_label = int(row['True_Label'])
        predicted_label = int(row[f"{model}_Prediction"])
        if true_label == label:
            return "blue" if predicted_label == true_label else
"crimson"
        else:
            return "none"
    except (KeyError, ValueError, TypeError):
        return "none"

# Generate map plots for selected labels
for label in [2, 7]:
    fig, axes = plt.subplots(1, 3, figsize=(20, 12))
    fig.suptitle(f"Predictions for Label {label} -
{label_names[label]}", fontsize=24, y=1.02)

    for i, model in enumerate(models):
        df_plot = merged_df.copy()
        df_plot["color"] = df_plot.apply(lambda row:
assign_colors_label(row, model, label), axis=1)
        df_plot = df_plot[df_plot["color"] != "none"]

        # Convert to GeoDataFrame and reproject
        gdf = gpd.GeoDataFrame(df_plot,
geometry=gpd.points_from_xy(df_plot.lon, df_plot.lat),
crs="EPSG:4326")
        gdf = gdf.to_crs("EPSG:27700")

        # Count correct and incorrect
        n_correct = (gdf['color'] == 'blue').sum()
        n_incorrect = (gdf['color'] == 'crimson').sum()

        # Plot UK boundary
        uk_boundary.plot(ax=axes[i], facecolor='lightgrey',
edgecolor='none', linewidth=0.2, alpha=0.4)

        # Plot prediction points
        gdf.plot(ax=axes[i], color=gdf["color"], markersize=46,
alpha=0.9)

        # Updated legend with counts
        correct_patch = mpatches.Patch(color='blue',
label=f'Correct ({n_correct})')
        incorrect_patch = mpatches.Patch(color='crimson',
label=f'Incorrect ({n_incorrect})')
        axes[i].legend(handles=[correct_patch, incorrect_patch],
loc='upper left', fontsize=28)

        # Add Scale Bar (in metres since EPSG:27700 is in meters)
        scalebar = ScaleBar(dx=1, units='m', location='lower
right', scale_loc='bottom', box_alpha=0.2, length_fraction=0.15)
        axes[i].add_artist(scalebar)

        # Add North Arrow
        axes[i].annotate('N', xy=(0.95, 0.08), xytext=(0.95,
0.05),
                         arrowprops=dict(facecolor='black',
width=5, headwidth=15),
                         ha='center', va='center', fontsize=16,
                         xycoords='axes fraction')

        # Set subplot title
        axes[i].set_title(f"{model} Predictions", fontsize=32)
        #axes[i].set_xlabel("Easting")
        #axes[i].set_ylabel("Northing")

    plt.tight_layout(rect=[0, 0, 1, 0.99])  # Adjust to make
space for overall title
    plt.show()
```